

JEPA Policy: Diffusion-Free Imitation Learning via Paired Action and Future Representation Prediction

Abstract—Standard behavior cloning supervises actions without explicitly constraining the future representation paired with each demonstrated action chunk. We introduce JEPA Policy, a diffusion-free framework that uses the action chunk and its observed future representation as paired training targets. Their tokens remain mutually visible in a shared Transformer and are refined through two forward passes, so future prediction directly shapes the action-generating representation. Dual-branch and gradient-routing controls show that the gain comes from this shared topology, not from attaching an auxiliary future-prediction head. Across eight simulated tasks, JEPA Policy improves its action-only parent and outperforms Diffusion Policy under the evaluated configurations, with only 0.29 ms additional model latency. A five-task, 630-episode physical-robot study preserves the pooled method ordering. Further audits find no complete representation collapse under action supervision and show that future-prediction error ranks failures within tasks. These results support paired future-representation supervision as a practical way to improve low-latency visuomotor imitation without iterative generative sampling.

Index Terms—Imitation learning, joint-embedding predictive architecture (JEPA), predictive representation learning, robot learning, robotic manipulation.

I. INTRODUCTION

ROBOT imitation learning aims to learn policies capable of completing complex manipulation tasks from expert demonstrations. Driven by advances in visual encoders, Transformer architectures, and generative action modeling, imitation learning has achieved remarkable progress on tasks such as grasping, pushing, insertion, drawer opening, and long-horizon tabletop manipulation. Typical behavior cloning methods, the paradigm since ALVINN [1], formulate policy learning as a supervised problem of predicting the expert action sequence $\mathbf{a}_{t:t+H}$ directly from the current observation $\mathbf{o}_t$. This paradigm is simple and efficient, but it focuses primarily on the action labels themselves and lacks explicit constraints on the future outcomes caused by the expert actions, and it inherits the compounding error of open-loop supervised imitation under the distribution shift its own actions induce [2].

In robotic manipulation, however, expert actions are not isolated labels. An action sequence is valid precisely because it advances the current state toward some task-relevant future state—an object pushed to the target region, a gripper approaching an effective grasp pose, or a tool forming stable contact with the target object. An ideal imitation learning policy should therefore not only learn “what the expert did” but also be constrained by “what future that action leads to.” In other words, an expert demonstration segment contains not only action supervision but also an implicit future-aware supervision signal corresponding to the consequences of the action. Standard behavior cloning typically uses only the former and ignores the latter.

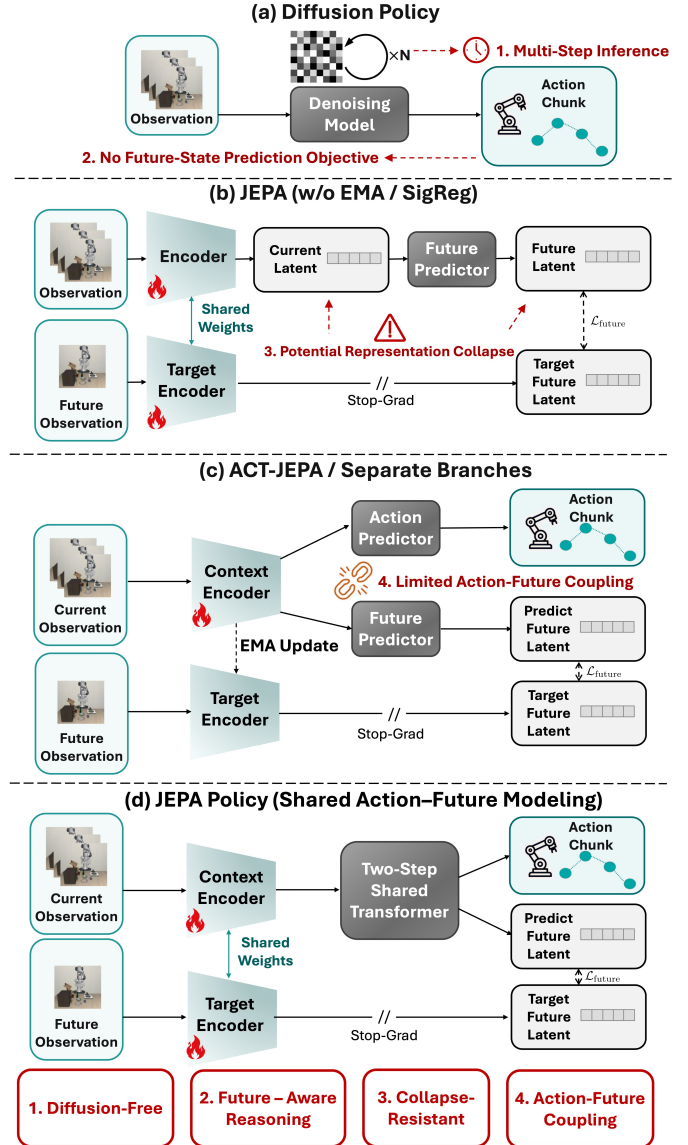


Fig. 1. Schematic comparison of design paradigms (architecture only, not a performance comparison). Red numbers mark the limitation each prior design carries; the four boxes along the bottom are JEPA Policy’s corresponding answers. Flames mark encoders trained by gradient descent—all of them from scratch, since no pretrained or frozen visual backbone is used anywhere in this paper—and “//” marks the stop-gradient.

Future prediction, however, is not an ordinary auxiliary task that can simply be appended to behavior cloning. If future observations or future observation representations are regressed from the current observation alone, the same scene may admit multiple plausible manipulation directions, grasp styles, or intermediate states, and naive regression tends to produce

averaged future targets inconsistent with the specific expert action. The key observation of this paper is that in expert demonstration data, action labels and future observations are not sampled independently; they come from the same trajectory. Given a sample $(o_t, \mathbf{a}_{t:t+H}, o_{t+k})$, the expert action $\mathbf{a}_{t:t+H}$ and the future observation representation $E_{\text{rgb}}(o_{t+k})$ form a natural pair. We therefore do not need to model the full distribution of all possible futures under the current observation; instead, future prediction can be constrained to the paired future representation that co-occurs with that expert action segment.

Diffusion policies provide a powerful generative action modeling paradigm for robot imitation learning. Through noise injection and denoising-style supervision, they learn smooth and expressive action prediction models and perform well on many visuomotor tasks. However, these methods typically require multi-step sampling or multiple network forward passes, incurring additional inference overhead, reducing the control frequency, and introducing non-negligible latency in real-time robotic systems. Recent work further suggests that the advantage of diffusion or flow policies in imitation learning does not necessarily come from explicit multi-modal distribution modeling, but may instead come from supervised iterative computation and noise injection [3]. Motivated by this, we adopt a coarse-to-refined two-step prediction process in the style of the *minimum iterative policy* (MIP) [3]: the first step produces coarse actions and future representations, and the second step refines them near the expert demonstration manifold. This design retains the low-latency advantage of few-step prediction while incorporating future-aware paired supervision into policy learning.

Based on these observations, we propose **JEPA Policy**, a diffusion-free framework for joint action–future observation representation prediction (see Fig. 1 for a paradigm comparison with representative designs). Fig. 1 places the design against three reference points. Diffusion Policy [4] (a) denoises iteratively, so one decision costs N network calls and no term in its objective constrains the future its action leads to. A plain latent-future JEPA [5] (b), trained without action supervision and without the usual anti-collapse devices—an EMA target encoder or variance–covariance regularization—is free to collapse its representation. ACT-JEPA [6] (c) shares a context encoder but routes it to *separate* predictor and decoder branches behind an EMA target encoder, so the action and future paths stay loosely coupled. JEPA Policy (d) places action and future representation tokens in one shared two-step Transformer where they are mutually visible; the future target is produced by the same encoder under shared weights with a stop-gradient, and no EMA or frozen *target* encoder is used.

Given the current observation, the model simultaneously predicts the expert action sequence and its paired future observation representation within one shared Transformer. Action and future-representation tokens share one self-attention stack and remain mutually visible in every layer, allowing the future objective to shape the action-generating representation directly.

From the perspective of supervision, JEPA Policy exploits each demonstration segment more fully than standard behavior cloning. Conventional behavior cloning extracts only the

expert action label from a segment, whereas our method simultaneously extracts the expert action and the future observation representation produced along the same trajectory. Each demonstration sample thus provides supervision not only on “what the expert did” but also on “what task-relevant future that action drives the system toward.” Compared with reconstructing future images, predicting the future in representation space prevents the model from wasting capacity on low-level visual details such as texture, illumination, and background that are weakly related to control, and focuses it on manipulation-relevant information such as object positions, contact relations, task progress, and action consequences.

The future prediction branch of JEPA Policy also provides a deployment-time interface. At inference, the policy outputs the action sequence through the two-step predictor and simultaneously produces a prediction of the future observation representation. After the action is executed, the actual future observation can be mapped into representation space by the same encoder and compared with the previously predicted future representation. This future-consistency error serves as a rollout-level diagnostic signal for analyzing whether the policy is drifting away from the expert demonstration manifold, entering high-risk states, or requiring human intervention. We do not equate this signal with a full uncertainty estimate; rather, it is a behavior-consistency diagnostic that arises naturally from future-aware policy learning.

We emphasize that JEPA Policy is fundamentally different from purely self-supervised JEPA-style latent prediction. In pure self-supervised representation prediction, without additional anti-collapse mechanisms the model may map different inputs to similar or identical representations, forming a trivial solution. Many self-supervised JEPA or Siamese representation learning methods therefore require an EMA target encoder, a frozen target encoder, architectural asymmetry, or explicit variance–covariance regularization. In contrast, the future observation representation prediction of JEPA Policy is always optimized jointly with expert action prediction. If the shared representation collapses, object positions, contact states, and control requirements under different observations become indistinguishable, directly increasing the action prediction error. Action supervision thus provides a task-driven anti-collapse constraint for future observation representation learning, encouraging the model to retain information useful for robotic manipulation without additionally introducing an EMA target encoder, a separately frozen encoder, or explicit variance–covariance regularization; the corresponding representation diagnostics will be verified in the experiments.

The contributions of this paper are fourfold:

- We propose **JEPA Policy**, a future-aware robot imitation learning framework based on paired action–future observation representation supervision. Unlike standard behavior cloning, which extracts only the expert action label from a demonstration segment, JEPA Policy simultaneously exploits the expert action sequence and the actual future observation representation from the same trajectory segment, so that each sample provides supervision on both “what the expert did” and “what future the action drives the system toward.” Architecturally, both token types occupy

one shared Transformer and remain mutually visible in every self-attention layer, so the future objective directly shapes the action-generating representation.

- We extend **MIP-style coarse-to-refined prediction** to the joint generation of actions and future observation representations. Inspired by recent findings on the source of gains in diffusion/flow policies, we do not reduce the core problem to explicit multi-modal modeling; instead, a two-step diffusion-free predictor refines the action sequence and its paired future observation representation. Because the future representation target comes from the same demonstration segment as the expert action, the model does not need diffusion-style sampling over all possible futures, enabling efficient action generation and future representation prediction.
- We use expert action supervision to provide a **task-driven anti-collapse constraint** for latent-space future observation representation prediction. Unlike purely self-supervised JEPA objectives, the shared representation of JEPA Policy must simultaneously support action prediction and future representation prediction; once the representation collapses, the action error immediately increases. Our method therefore needs neither an EMA target encoder nor a separately frozen encoder, and it uses no explicit variance–covariance regularizer.
- We propose a **deployment-time future-consistency diagnostic**. The future observation representation predicted by the policy can be compared with the actual future observation representation after execution, forming a rollout-level future-consistency error usable for failure analysis, risk monitoring, or potential human intervention.

II. RELATED WORK

A. Imitation Learning for Robotic Manipulation

Behavior cloning trains policies to match expert actions via supervised learning. Modern visuomotor imitation learning methods typically combine image encoders, Transformers, and action chunking to model temporally continuous action sequences [7], [8]. The key advantage of these methods is that inference requires only a single direct forward pass. However, their training objectives focus on action matching and do not explicitly require the policy representation to encode the future outcomes produced by the demonstrated actions.

B. Diffusion Policy and Generative Action Modeling

Diffusion Policy models action generation as an iterative denoising process conditioned on observations [4]. This approach is widely regarded as having strong modeling capacity for complex action distributions and performs well across multiple visuomotor control tasks. Meanwhile, recent work has noted that the gains of diffusion or flow-matching policies may also stem from the optimization and smoothing effects of noise injection and denoising training, rather than solely from multi-modal modeling itself [3]. The primary cost is that inference requires multiple network calls. Diffusion was not the first answer to multi-modal action distributions: implicit behavior cloning fits an energy-based policy instead of a regressor [9],

and Behavior Transformers discretize the action space and predict a residual offset [10]. Our setting differs from all of these in what the extra supervision is for—not modelling the spread of possible actions, but binding the action to the one future it actually produced.

Few- and one-step alternatives already reduce sampling cost through consistency or flow reformulations [11]–[14], while DP3 [15] reduces observation cost. We do not benchmark against them and make no relative speed claim. Our predictor is not a new accelerator; it adds paired future supervision within an existing two-pass inference budget.

C. Predictive Representation Learning and JEPA

Joint-embedding predictive architectures learn by predicting the latent representation of a target view or future observation rather than reconstructing pixels [5], [16]. This avoids fully modeling low-level image details but requires handling representation collapse. Existing methods commonly use architectural asymmetry, stop-gradient, momentum encoders, or variance–covariance regularization [17]–[19]. Our setting differs: the predictive representation is embedded inside an imitation learning policy, and the shared representation must support accurate expert action prediction. Most related to this paper is ACT-JEPA [6], which also brings JEPA into imitation learning by jointly predicting action sequences and latent representations of future observations. Its architecture, however, routes a shared context encoder to *separate* branches—an observation predictor P_ϕ and an action decoder D_τ —which are not mutually visible within one attention stack, and it relies on an EMA target encoder to prevent collapse. JEPA Policy instead uses one mutually visible stack, a stopped target as in SimSiam [20], and action supervision rather than an EMA or frozen target encoder. Because non-contrastive objectives can fail through partial rank collapse [21], [22], Sec. IV-I audits the representation spectrum. GR-1 [23] and Seer [24] also predict future visuals and actions in one Transformer. JEPA Policy instead uses a paired latent target, no video pretraining, two fixed feed-forward steps, and a shared-stack topology tested in Sec. IV-D.

A parallel line scales imitation with multi-task or pre-trained VLA backbones, including RT-1 [25], RT-2 [26], OpenVLA [27], Octo [28], Open X-Embodiment [29], and π_0 [30]. Concurrent VLA-JEPA [31] predicts leakage-free future latents with a pretrained VLM and flow matching, while JEPA-VLA [32] injects video-predictive embeddings into existing VLAs. JEPA Policy instead trains on each task’s demonstrations, shares action and future tokens in one stack, uses two MIP steps, and relies on action loss against collapse.

D. World Models and Future Prediction

World and forward models predict action-conditioned futures for planning or model-predictive control [33]. V-JEPA 2-AC [34] uses a frozen video-pretrained encoder and latent predictor for image-goal MPC. CURL [35], SPR [36], DreamerV3 [37], and TD-MPC2 [38] likewise learn representations or dynamics for rollout. JEPA Policy learns no separate dynamics model, performs no planning rollout, and reconstructs no future

images. It consumes one demonstrated future latent paired with the supervised action as a policy-training signal.

E. Runtime Failure Monitoring for Learned Policies

Detecting at run time that a learned policy is failing is an established problem, and it inherits a large literature on predictive uncertainty: softmax-confidence baselines for out-of-distribution inputs [39], deep ensembles [40], and, where a formal guarantee is wanted, conformal prediction, which has been used to give distribution-free safety margins for planning among dynamic agents [41]. Sentinel [42] monitors generative policies by combining a statistical measure of temporal action consistency with a vision–language model that judges task progress, and FAIL-Detect [43] formulates uncertainty-aware failure detection for imitation policies without requiring failure data. Relative to these, the diagnostic in this paper is deliberately narrower: it is not a purpose-built monitor but a by-product of the training objective—the policy already predicts the future representation, so comparing that prediction with the realized one costs one encoder forward and no extra model. We therefore position it as a per-task-calibrated risk signal rather than a competitor to dedicated monitors, and we note that our audit reports no comparison against them; that comparison is the natural next step for this contribution.

III. METHOD

This section specifies JEPA Policy’s paired objective, shared stack, two-step training rule, and deployment diagnostic.

A. Problem Formulation

Given an expert demonstration dataset

$$\mathcal{D} = \{(o_t, \mathbf{a}_{t:t+H}, o_{t+k})\}, \quad (1)$$

where o_t denotes the current observation or observation history window, $\mathbf{a}_{t:t+H}$ denotes the expert action sequence over a prediction horizon of H , and o_{t+k} denotes the future observation at horizon k . Both H and k are counted in environment control steps; their wall-clock meaning is quantified in Sec. IV-A. Here k controls the temporal span of the future observation relative to the present: if k is too small, $\mathbf{z}_{t+k}$ degenerates toward the current observation and the future information becomes trivial; if k is too large, the current action chunk can hardly determine that future and the supervision from the paired target is weakened. We therefore set k on the same order of magnitude as the action horizon H and determine its concrete value by ablation. Conventional direct policy learning formulates imitation learning as a supervised mapping from the current observation to expert actions:

$$\pi_\theta(o_t) \rightarrow \hat{\mathbf{a}}_{t:t+H}. \quad (2)$$

This formulation requires the policy only to predict expert actions, without explicitly constraining the future outcomes that action sequence induces in the demonstration trajectory.

JEPA Policy learns the following joint mapping:

$$\pi_\theta(o_t) \rightarrow \{\hat{\mathbf{a}}_{t:t+H}, \hat{\mathbf{z}}_{t+k}\}, \quad (3)$$

where $\hat{\mathbf{a}}_{t:t+H}$ is the predicted action sequence and $\hat{\mathbf{z}}_{t+k}$ is the predicted future observation representation. Here $\pi_\theta(o_t)$ is a shorthand for the final composite policy. For a unified notation, we write the policy network as a coarse-to-refined two-step predictor with explicit inputs, $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, s, t)$: it takes the current observation, a (zero or noisy) action input, a future observation representation input, and the two endpoints (s, t) of the interval the step operates on, and outputs the predicted action and future observation representation; internally it is the shared Transformer policy described in Sec. III-B— F_θ and that shared policy are the same network, differing only in notation. The future observation representation target is obtained by passing the actual future observation through the observation encoder:

$$\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k}), \quad (4)$$

where $E_{\text{rgb}}(\cdot)$ is the image sub-path of the same observation encoder E : the per-camera backbone features, globally pooled and concatenated, taken *before* the fusion layer that combines them with the low-dimensional inputs. The conditioning representation $\mathbf{c}_t = E(o_t)$ is the output of that fusion layer and therefore also carries proprioception, whereas the future target is image-only and higher-dimensional (1024 against 384 in our configuration). Writing $\mathbf{z}_t = E_{\text{rgb}}(o_t)$ for that same sub-path applied to the *current* observation, the two are related by $\mathbf{c}_t = \text{Fuse}([\mathbf{z}_t, \mathbf{s}_t])$, where $\mathbf{s}_t$ collects the low-dimensional (proprioceptive) inputs: the image latent is the part the two observations share, and the condition adds proprioception on top of it. Fig. 2 therefore draws $\mathbf{z}_t$ and $\mathbf{z}_{t+k}$ —the same function at two times—and leaves the fusion step implicit. The two are computed by one encoder with one set of parameters—in particular the same image backbone—but they are not the same function of the observation, and we distinguish them by notation throughout. Unlike future image reconstruction, JEPA Policy predicts future observation representations only in latent space, avoiding the modeling of low-level visual details such as texture, illumination, and background that are weakly related to robot control.

The shared encoder E is trained end to end, without a frozen or EMA target encoder. We stop gradients only through the future target $\mathbf{z}_{t+k}$; $\mathcal{L}_{\text{future}}$ still updates E through the current-observation path $\mathbf{c}_t$, as detailed in Sec. III-C.

B. Shared Action–Future Observation Representation Policy

The overall structure of JEPA Policy is shown in Fig. 2. The policy is exactly the $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, s, t)$ introduced in Sec. III-A, consisting of an observation encoder, H action input tokens, a single future input token, a shared Transformer, an action prediction head, and a future observation representation prediction head. The current observation is first encoded into a conditioning representation:

$$\mathbf{c}_t = E(o_t). \quad (5)$$

Together with an embedding of the two interval endpoints (s, t) of the current step, $\mathbf{c}_t$ forms the conditioning memory

$$\mathbf{M} = \text{Enc}([\text{TimeEmb}(s, t), \text{Embed}_o(\mathbf{c}_t)]), \quad (6)$$

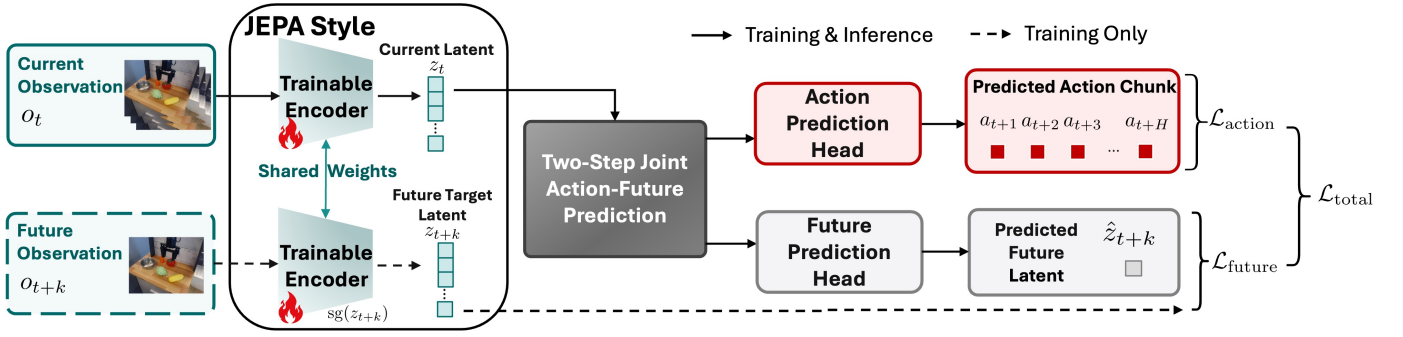


Fig. 2. Overall architecture of JEPA Policy. A single shared encoder is applied to both observations; $\mathbf{z}_t$ and $\mathbf{z}_{t+k}$ are its image sub-path at the two times. The Transformer is conditioned on $\mathbf{c}_t$, which fuses $\mathbf{z}_t$ with the low-dimensional inputs (Sec. III-B); that step is left implicit here. Action input tokens $\mathbf{q}_a$ and future representation tokens $\mathbf{q}_z$ enter one shared Transformer T_θ , where they are *mutually visible* in every self-attention layer. The two heads then read out the predicted action sequence $\hat{\mathbf{a}}_{t:t+H}$ and the predicted future observation representation $\hat{\mathbf{z}}_{t+k}$. The target future observation representation $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$ is produced by the image sub-path of the same shared encoder, with stop-gradient (sg). This mutual visibility, absent in separate-branch designs, lets the future representation prediction objective shape the action representation.

which the shared Transformer reads by cross-attention. The action input $\mathbf{a}_{\text{in}}$ and the future observation representation input $\mathbf{z}_{\text{in}}$ are embedded by two separate linear projections and placed at $H + 1$ positions of one shared sequence, with a learned positional embedding $\mathbf{p}$ over all positions and a learned type embedding $\mathbf{e}_z$ marking the single future position:

$$\mathbf{q}_a = \text{Embed}_a(\mathbf{a}_{\text{in}}) + \mathbf{p}_{1:H}, \quad (7)$$

$$\mathbf{q}_z = \text{Embed}_z(\mathbf{z}_{\text{in}}) + \mathbf{e}_z + \mathbf{p}_{H+1}. \quad (8)$$

The step parameters are deliberately *not* added to these tokens; they enter only through $\mathbf{M}$ in Eq. (6). The shared Transformer then jointly models the two token types:

$$\mathbf{Y} = T_\theta([\mathbf{q}_a, \mathbf{q}_z], \mathbf{M}), \quad (9)$$

where T_θ denotes the shared policy Transformer (the backbone of F_θ), $[\cdot, \cdot]$ denotes token concatenation, and each of its layers applies unmasked self-attention over the $H + 1$ concatenated tokens followed by cross-attention to $\mathbf{M}$. The output $\mathbf{Y}$ is split into action token representations $\mathbf{Y}_a$ and future token representations $\mathbf{Y}_z$:

$$\mathbf{Y}_a, \mathbf{Y}_z = \text{split}(\mathbf{Y}). \quad (10)$$

The two prediction heads then output the action sequence and the future observation representation, respectively:

$$\hat{\mathbf{a}}_{t:t+H} = h_a(\mathbf{Y}_a), \quad (11)$$

$$\hat{\mathbf{z}}_{t+k} = h_z(\mathbf{Y}_z), \quad (12)$$

where $h_a(\cdot)$ denotes the action prediction head and $h_z(\cdot)$ denotes the future observation representation prediction head. The overall mapping is exactly $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, s, t) = (\hat{\mathbf{a}}_{t:t+H}, \hat{\mathbf{z}}_{t+k})$. The two steps are run on the interval endpoints $(s, t) = (0, \tau)$ and then $(\tau, 1)$. In the first step $\mathbf{a}_{\text{in}} = \mathbf{0}$ and $\mathbf{z}_{\text{in}} = \mathbf{0}$, so the tokens reduce to the learned biases of the two input projections plus $\mathbf{p}$ and $\mathbf{e}_z$, i.e., they degenerate into a set of learned queries and the step is equivalent to regressing a coarse prediction directly from the current observation; the second step takes the noised expert targets at training time, and the coarse predictions of the first step at inference, as its input tokens, and performs manifold refinement.

The key property of this structure is that future observation representation prediction is not an isolated auxiliary branch appended after the policy. The action tokens and the future token are jointly modeled within the same Transformer through shared attention, allowing future observation representation prediction to participate in shaping the shared policy representation on which action generation relies. Concretely, the action tokens and the future token reside in the same self-attention stack, which is applied without any causal, token-type or memory mask, so they are mutually visible in every layer, so the representation $\mathbf{Y}_a$ read by the action head has already aggregated information from the future representation tokens during the forward pass; the gradient of the future prediction loss also backpropagates through these shared attention layers and rewrites the intermediate representation on which the action branch relies. In contrast, architectures that merely attach independent prediction branches after a shared encoder (such as ACT-JEPA [6]) let the future loss reach the shared encoder but keep the two branches mutually invisible: the future prediction never enters the computation that produces the action tokens, so it can shape the action pathway only indirectly, through the encoder they happen to share—this structural difference is exactly what the shared vs. separate-head ablation in Sec. IV-D is designed to test. In other words, JEPA Policy learns not only “what the expert did” but also “what future outcome the expert actions correspond to.”

C. Joint Action–Future Representation Objective

The action prediction loss is defined as

$$\mathcal{L}_{\text{act}} = \langle (\hat{\mathbf{a}}_{t:t+H} - \mathbf{a}_{t:t+H})^2 \rangle, \quad (13)$$

where $\langle \cdot \rangle$ denotes the mean over the action dimension, the horizon H and the batch, so that the loss scale is independent of H and of the action dimensionality. The future observation representation prediction loss aligns the predicted future representation with the target representation of the actual future observation:

$$\mathcal{L}_{\text{future}} = d(\hat{\mathbf{z}}_{t+k}, \text{sg}(\mathbf{z}_{t+k})), \quad (14)$$

where $d(\cdot, \cdot)$ is a distance measure in latent space; by default we use a **normalized MSE**—the per-sample prediction error is first scaled by the RMS magnitude of the target embedding before squaring—which decouples the latent-space scale from the action-space scale and makes the two loss weights easier to balance (see Sec. IV-A for implementation details). $\text{sg}(\cdot)$ denotes a stop-gradient that we **always** apply to the future target representation $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$. Because the current and future observations use the **same shared encoder parameters** E , $\mathcal{L}_{\text{future}}$ **still updates the encoder**: the prediction $\hat{\mathbf{z}}_{t+k}$ depends on the current conditioning representation $\mathbf{c}_t = E(o_t)$, and gradients flow back to E through this current-observation path. The stop-gradient cuts only the future-observation target path $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$, preventing direct target-side co-adaptation but not, by itself, excluding a constant representation. The future loss still shapes the encoder through the current-observation path, while the action loss $\mathcal{L}_{\text{act}}$ supplies the task-driven constraint tested in Sec. IV-I. In implementation, both the action loss and the future observation representation loss are averaged over the batch and output dimensions.

The overall training objective is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{act}} + \lambda_z \mathcal{L}_{\text{future}}, \quad (15)$$

The coefficients λ_a and λ_z weight these two terms.

Two settings of λ_z are implemented. In *fixed* mode it is a constant. Every experiment reported in this paper uses the *adaptive ratio* mode, in which λ_z is recomputed at each step so that the weighted future term holds a target ratio ρ to the action term:

$$\lambda_z = \text{clip}\left(\rho \frac{\text{sg}(\mathcal{L}_{\text{act}})}{\text{sg}(\mathcal{L}_{\text{future}}) + \varepsilon}, \lambda_{\min}, \lambda_{\max}\right), \quad (16)$$

with $\varepsilon = 10^{-8}$. Both losses enter detached, so λ_z carries no gradient and Eq. (16) only rescales the future term. We use $\lambda_a = 1$ throughout, $\rho = 0.10$ by default (ablated over $\{0.05, 0.10, 0.20\}$ in Sec. IV-F), and clip bounds $\lambda_{\min} = 10^{-6}$, $\lambda_{\max} = 0.1$. One consequence should be stated explicitly, because ρ indexes every ablation table: in this configuration $\lambda_{\max}$ coincides with ρ , so the upper clip is active whenever $\mathcal{L}_{\text{act}} > \mathcal{L}_{\text{future}}$, and in that regime the realized ratio falls strictly below ρ . The target ratio is therefore an upper bound on the future term’s share of the objective rather than a value that is exactly maintained.

Through Eq. (15), action prediction and future observation representation prediction are jointly optimized in the same policy network, introducing paired action–future supervision for diffusion-free imitation learning.

D. MIP-Style Two-Step Prediction Training

To avoid the inference overhead of the long-chain multi-step denoising in diffusion policies, JEPA Policy adopts a MIP-style two-step prediction process. The two key ingredients of MIP are **supervised iterative computation** and **stochasticity injection**: both prediction steps are supervised, and the input of the second step carries noise. Unlike flow or diffusion, MIP does not learn a continuous velocity field that requires multi-step integration; it **directly regresses the target** (actions or

Two-Step Joint Action-Future Prediction

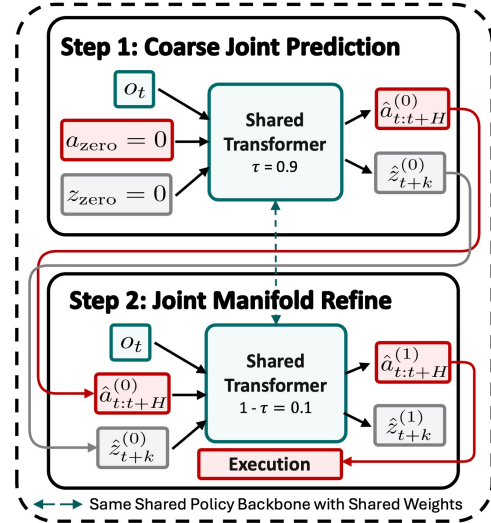


Fig. 3. Coarse-to-refined MIP-style training for joint action and future observation representation prediction. The first step maps a zero entry (time 0) to a coarse prediction; the second step refines it (time τ). The figure draws the *inference* path, in which the second step consumes the coarse output of the first; at training time the second-step input is instead the noised expert target defined in Sec. III-D. Both steps are supervised, with residuals scaled by $\frac{1}{\tau}$ and $\frac{1}{1-\tau}$. No diffusion-style iterative sampling is used.

future observation representations) at both steps and applies scale normalization to the two-step residuals. We apply this process simultaneously to the expert action sequence and the future observation representation. The two-step interpolation parameter τ is a fixed hyperparameter ($\tau \in (0, 1)$, default $\tau = 0.9$) that takes the same value during training and inference, and it is unrelated to the time-step index t used in subscripts such as o_t and $\mathbf{a}_{t:t+H}$.

The first step takes zero vectors as input at time index 0 and directly produces coarse actions and coarse future observation representations from the current observation:

$$(\hat{\mathbf{a}}_{t:t+H}^{(0)}, \hat{\mathbf{z}}_{t+k}^{(0)}) = F_{\theta}(o_t, \mathbf{0}, \mathbf{0}, 0, \tau). \quad (17)$$

The first-step loss is

$$\mathcal{L}^{(0)} = \lambda_a \left\| \frac{\hat{\mathbf{a}}_{t:t+H}^{(0)} - \mathbf{a}_{t:t+H}}{\tau} \right\|_2^2 + \lambda_z \left\| \frac{\hat{\mathbf{z}}_{t+k}^{(0)} - \text{sg}(\mathbf{z}_{t+k})}{\tau} \right\|_2^2. \quad (18)$$

The second step constructs noisy inputs near the expert target. Given Gaussian noise $\epsilon_a, \epsilon_z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, define

$$\tilde{\mathbf{a}}_{t:t+H} = \mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a, \quad (19)$$

$$\tilde{\mathbf{z}}_{t+k} = \mathbf{z}_{t+k} + (1 - \tau)\epsilon_z, \quad (20)$$

and the model refines them back to the expert action and the true future observation representation at time index τ :

$$(\hat{\mathbf{a}}_{t:t+H}^{(1)}, \hat{\mathbf{z}}_{t+k}^{(1)}) = F_{\theta}(o_t, \tilde{\mathbf{a}}_{t:t+H}, \tilde{\mathbf{z}}_{t+k}, \tau, 1). \quad (21)$$

The second-step loss is

$$\mathcal{L}^{(1)} = \lambda_a \left\| \frac{\hat{\mathbf{a}}_{t:t+H}^{(1)} - \mathbf{a}_{t:t+H}}{1 - \tau} \right\|_2^2 + \lambda_z \left\| \frac{\hat{\mathbf{z}}_{t+k}^{(1)} - \text{sg}(\mathbf{z}_{t+k})}{1 - \tau} \right\|_2^2. \quad (22)$$

The two-step residuals are divided by τ and $1 - \tau$, respectively, to put the two steps on a common scale. The second

step is the case the rescaling is derived from: its input deviates from the expert target by $(1 - \tau)\epsilon$, so dividing by $1 - \tau$ makes the model regress a unit-scale correction. The first step is not of that form—its input is exactly zero rather than a perturbed target—and its division by τ is the matching convention for an interval of length τ rather than a consequence of an input perturbation. The two steps are decoupled during training: the input of the second step is constructed from the ground-truth target and noise, not from the output of the first step. The two supervised prediction steps (iterative computation) and the input noise of the second step (stochasticity injection) together constitute the two effective ingredients of MIP. The final joint MIP objective is

$$\mathcal{L}_{\text{MIP}} = \mathbb{E}_{\epsilon} [\mathcal{L}^{(0)} + \mathcal{L}^{(1)}]. \quad (23)$$

Here the second-step input adopts the simplified form $\mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a$; when τ is close to 1 (e.g., 0.9), it is practically equivalent to the original form $\tau \mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a$. In addition, consistent with the default distance in Sec. III-C, the future observation representation residuals in both steps are first normalized by the RMS scale of the target embedding before squaring in the implementation; this normalization factor is omitted in the equations above for clarity.

Fig. 3 illustrates the two-step MIP-style joint training procedure. At inference, JEPA Policy first evaluates the zero-input coarse step above with the same fixed τ , then supplies both coarse outputs to a noise-free refinement:

$$(\hat{\mathbf{a}}_{t:t+H}^{(1)}, \hat{\mathbf{z}}_{t+k}^{(1)}) = F_{\theta} \left(o_t, \hat{\mathbf{a}}_{t:t+H}^{(0)}, \hat{\mathbf{z}}_{t+k}^{(0)}, \tau, 1 \right). \quad (24)$$

The refined action $\hat{\mathbf{a}}_{t:t+H}^{(1)}$ controls the robot. In the original MIP formulation, the second-step input is the first-step output scaled by τ , i.e., $\tau \hat{\mathbf{a}}_{t:t+H}^{(0)}$; we adopt the simplified form above, and the two coincide as τ approaches 1.

E. Future Observation Representation Prediction under Paired Action Supervision

Predicting a future observation representation from the current observation alone is under-constrained. Similar observations may precede different futures, so plain regression may average targets or encode control-irrelevant variation.

JEPA Policy instead uses the demonstration tuple of Sec. III-A: the action target and $\mathbf{z}_{t+k}$ from the same segment jointly supervise the two outputs. At the supervision level, each segment supplies both labels; at the representation level, the future loss reaches the action representation through the shared attention stack. The target is therefore the particular demonstrated outcome paired with the action, not an average over hypothetical futures.

This construction assumes neither a deterministic one-to-one action–future mapping nor a model of the full future distribution. The two-step predictor of Sec. III-D produces both outputs; the future branch shapes the policy during training and supplies the diagnostic defined next at deployment.

F. Deployment-Time Future-Consistency Diagnostics

At inference, JEPA Policy outputs not only the action sequence used for control but also the predicted future observation representation $\hat{\mathbf{z}}_{t+k}$. After the policy executes the action, the actual future observation o_{t+k}^{real} becomes available, and the same observation encoder produces its representation:

$$\mathbf{z}_{t+k}^{\text{real}} = E_{\text{rgb}}(o_{t+k}^{\text{real}}). \quad (25)$$

We accordingly define the rollout-level future-consistency error:

$$e_{\text{fc}}(t, k) = d(\hat{\mathbf{z}}_{t+k}, \text{sg}(\mathbf{z}_{t+k}^{\text{real}})). \quad (26)$$

If the policy’s actions remain near the task-relevant manifold covered by the training demonstrations, the predicted future representation should agree with the actual post-execution future representation; conversely, a large e_{fc} may indicate that the policy behavior deviates from expectation, that a contact outcome is abnormal, or that the visual state has moved outside the training distribution. We treat this quantity as a diagnostic signal for failure analysis and human-intervention triggering, not as a strict probabilistic uncertainty estimate. In practical deployment, the alarm threshold can be chosen using a sliding-window average $\bar{e}_{\text{fc}}$, task-phase-conditioned thresholds, or ROC curves computed from successful/failed validation rollouts.

IV. EXPERIMENTS

We evaluate JEPA Policy on eight simulated manipulation tasks from three benchmark suites, with three training seeds per configuration, and complement the success-rate comparison with two dedicated audits: a pre-specified representation-collapse audit over 84 checkpoints (Sec. IV-I) and a 5,200-episode rollout audit of the future-consistency diagnostic (Sec. IV-J). Runs that did not reach the full protocol at reporting time are marked in the corresponding captions. Sec. IV-K repeats the comparison on physical hardware.

A. Setup

Tasks. We use LIBERO [44] (MokaMoka, MugMug), robomimic [8] (Square PH, Tool Hang PH, and Transport PH with four cameras) and MimicGen [45] (Coffee Preparation D1, Kitchen D1, Three Piece Assembly D1); all three are built on robosuite/MuJoCo [46], [47]. The demonstration sets differ widely in size: 50 demonstrations per LIBERO task (20,794 and 12,909 transitions), 200 per robomimic task (30,154 for Square, 95,962 for Tool Hang, 93,752 for the four-camera Transport set), and 1,000 per MimicGen task (334,869 to 687,674 transitions). The four-camera Transport dataset (four 84×84 RGB views) was generated for this work and passed an independent structural audit; a two-camera variant exists for one seed and is excluded from the main comparison.

Protocol. Every run trains for 300k gradient steps (batch size 256, learning rate 10^{-4} , EMA rate 0.995) and is evaluated every 10k steps with 40 rollout episodes under a fixed evaluation seed, i.e., 30 evaluations per run. We report the maximum success rate over the 30 evaluations (best-checkpoint protocol), averaged over seeds {41, 42, 43}. With 40 episodes per evaluation the single-evaluation granularity is 2.5 points and the three-seed

mean resolution is ≈ 0.83 points; differences within one to two granularity steps should not be over-interpreted.

Timing. All environments run at 20 Hz control. The policy predicts an action chunk of $H=10$ steps on the robomimic and MimicGen tasks and $H=16$ on the two LIBERO tasks, and in both cases executes the first 8 before re-planning, i.e., actions are issued at 20 Hz with a policy decision rate of 2.5 Hz. The future horizon k counts environment transitions after the newest observation frame: the default $k=4$ target lies 0.2 s ahead ($k=2/6/8$: 0.1/0.3/0.4 s), i.e., within the 0.4 s span of the currently executed action chunk.

Implementation details. The observation encoder E is a ResNet-18 [48] image encoder trained jointly with the policy by default; a frozen DINOv2 [49] ViT-S/14 encoder (CLS token) is supported as an alternative backbone. The future observation representation target is a single pooled global embedding (rather than patch tokens) of the future observation produced by the same encoder, using only the image-feature channels, taken at the input of the encoder’s fusion layer rather than at its output, so that low-dimensional inputs such as proprioception are excluded; stop-gradient is applied. To preserve the spatial correspondence between current and future observations under image augmentation, the two share the same random crop window during training (temporal-consistent crop). The future loss uses the target-RMS-normalized MSE; the two-step interpolation parameter is fixed to $\tau = 0.9$; the future horizon defaults to $k = 4$; inference uses the EMA weights. This exponential moving average is Polyak weight averaging of the trained policy for evaluation, applied to all methods including the baseline; it is not a BYOL-style EMA *target* encoder, which JEPA Policy does not use—the future target is produced by the live shared encoder with stop-gradient (Sec. III-B). The action loss weight is $\lambda_a = 1$, and the future loss weight follows the adaptive ratio rule of Eq. (16) with target ratio $\rho = 0.10$ and clip bounds $[10^{-6}, 0.1]$, ablated over $\rho \in \{0.05, 0.10, 0.20\}$. Peak device memory stays below 6 GiB per training run (Tool Hang: 5.28 GiB allocated/5.71 GiB reserved for JEPA Policy, versus 5.02 GiB allocated for MIP); inference peak activations are 362/355 MiB (JEPA/MIP; Table III).

B. Compared Methods

Table I contrasts the five arms in few-step inference, future-representation supervision, shared attention, and rollout diagnostics. Implementation details follow.

- **MIP (action-only):** the minimum iterative policy of [3], run from the released MIP implementation—which is also the codebase this work extends. Setting $\lambda_z = 0$ and removing the future tokens recovers that policy exactly, so this arm is at once a published baseline and the exact ablation of JEPA Policy: architecture, optimizer, data pipeline and evaluation harness are shared by construction rather than by reimplementing.
- **Diffusion Policy** [4]: the diffusion *transformer* policy at a 10-step action horizon, chosen so that the action-chunk length matches JEPA Policy on the robomimic and MimicGen tasks. It is trained under an aligned protocol—same tasks and seeds, 300k gradient steps, batch 256, and

the same evaluation budget of 30 rollout evaluations of 40 episodes—and its best `test/mean_score` follows the same best-checkpoint reading. Its sampler is DDPM with 100 inference steps, the setting it was trained with. Per task we report the better of two action parameterizations (absolute and delta), since the absolute-action variant failed to learn three robomimic tasks. One caveat belongs with this baseline: on the two LIBERO tasks JEPA Policy uses a 16-step horizon, so the horizon match is exact only outside LIBERO. Tool Hang is where this baseline is weakest (46.7); it is the one task in our suite that the robomimic authors label high-precision, and it is the setting in which an action-sequence policy is most exposed to accumulated pose error before the final insertion.

- **Dual-independent:** the conditioning path (observation embedding, condition encoder, future-memory adapter and the two interval embeddings; 1.40 M parameters) is shared and produces a *fixed* observation memory, which is then consumed by two parameter-independent decoder stacks—an action stack (18.93 M) and a future stack (4.75 M), each built from pre-norm self-attention, fixed-memory cross-attention and an FFN. No path connects the two stacks, so the future loss produces exactly zero gradient on the action stack and the action loss exactly zero on the future stack. No stop-gradient is placed on that shared path, so both losses train it: exactly as in a jointly trained separate-branch design, the future objective still shapes the shared representation, while staying invisible to the computation that produces the action. That is the ACT-JEPA property this ablation is built to isolate. What is deliberately not reproduced is everything that would confound it—the target is the visual $\mathbf{z}_{t+k}$ rather than a proprioceptive one, action supervision with a stopped target replaces the EMA target encoder, and the training recipe is ours—because published ACT-JEPA would confound the topology comparison.
- **Dual-cross:** identical to Dual-independent plus a bidirectional cross-attention block at every layer (5.35 M). Each layer applies self-attention on both branches, exchanges information through the cross block, then applies the memory-attention and FFN steps. The two branches are therefore mutually visible, but through an explicit cross module rather than by sharing one attention stack.
- For reference, JEPA Policy itself passes the concatenation of action tokens $\mathbf{q}_a$ and future tokens $\mathbf{q}_z$ through a single stack (20.27 M) under full bidirectional self-attention with no causal, token-type or memory mask. All three variants share the encoder (22.90 M), the losses, the MIP two-step schedule, the stop-gradient target and $\rho = 0.10$; totals are 43.96 M (JEPA Policy), 48.38 M (Dual-independent) and 53.73 M (Dual-cross).
- **SIGReg variant:** JEPA Policy plus an explicit isotropic-Gaussian regularizer on the encoder latents in the style of SIGReg [50] (weight 0.09), used as an explicit-regularization reference in the collapse analysis.
- $\lambda_a = 0$ **variant:** JEPA Policy with the action loss weight set to zero, so the future objective trains the shared encoder alone; the anti-collapse control of Sec. IV-I. It is trained

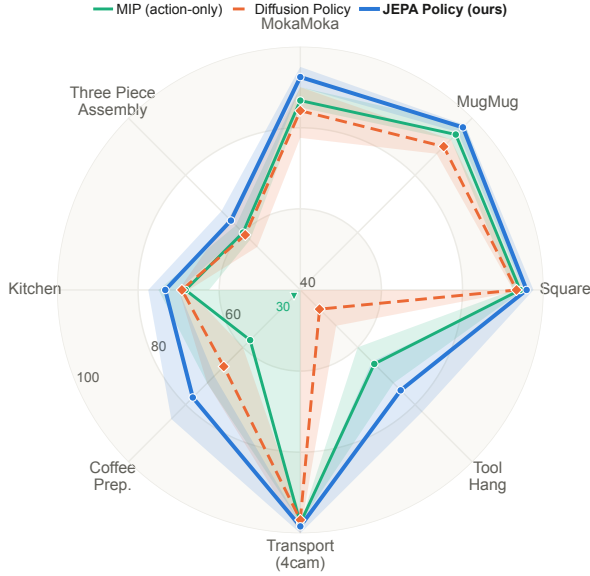


Fig. 4. Main comparison over eight tasks (line: three-seed mean best-checkpoint success; band: seed range; radial axis 40–100; means in Table II). A ▼ and printed value mark a seed below the axis floor.

on Square PH (seed 42) under an otherwise identical recipe and audited at its 220k-step EMA checkpoint.

TABLE I
METHOD PROPERTIES OF THE COMPARED APPROACHES.

Method	Few-step inference	Future repr. supervision	Shared attention	Diagnostic signal
MIP (action-only)	Yes (two-step)	No	—	No
Diffusion Policy	No	No	—	No
Dual-independent	Yes (two-step)	Yes (separate)	No	Optional
Dual-cross	Yes (two-step)	Yes (cross-attn)	No	Optional
JEPA Policy	Yes (two-step)	Yes (shared)	Yes	Yes

C. Main Results

Table II and Fig. 4 present the main comparison on the eight LIBERO/robomimic/MimicGen tasks. JEPA Policy improves over the MIP (action-only) arm on all eight tasks, with the largest gains on Coffee Preparation (57.5 → 77.5) and Tool Hang (65.8 → 75.0), and an eight-task mean improvement of +6.1 points (78.0 → 84.2). The Coffee Preparation baseline mean is depressed by an outlier seed (seed 42 reaches only 30.0 while seeds 41/43 reach 70.0/72.5); the JEPA Policy gain remains clear on every individual seed. Gains on Square, MugMug, and Transport are within one to two granularity steps and should be read as “no worse, slightly better” at their high base rates. Aggregated per seed—the small numbers in Table II—the weakest JEPA Policy seed (82.8 over the eight tasks) is still above the strongest seed of each of the other four arms (80.3 baseline, 79.4 dual-independent, 80.6 dual-cross, 77.2 Diffusion Policy), so the ordering does not depend on which seed is drawn.

Statistical reading. We treat the task as the unit of analysis, since the three seeds of a task are not independent of one another. Over the eight tasks the mean gain over the MIP (action-only) arm is +6.15 points with a paired t interval of $[+0.97, +11.32]$ ($t(7) = 2.81$) and a paired bootstrap interval of $[+2.81, +10.52]$ (20,000 resamples); the direction is positive on all eight tasks, giving an exact two-sided sign test and Wilcoxon signed-rank test of $p = 0.0078$. Against Dual-cross the corresponding figures are +5.94 points, $[+1.25, +10.63]$, eight of eight positive ($p = 0.0078$); against Dual-independent, +6.88 points, $[+0.71, +13.04]$, seven of eight positive (sign test $p = 0.07$, Wilcoxon $p = 0.023$; the exception is Transport). We deliberately do not report a seed-level test: with three seeds the paired statistic is $t(2) = 4.18$ against a critical value of 4.303, and no three-seed design can reach $p < 0.05$ under an exact two-sided sign-flip permutation, whose p -value floor is 0.25. The evidence for the effect is therefore its consistency across tasks, not its significance across seeds; Fig. 5 shows that it is also consistent across training time.

The gain does not depend on the best-checkpoint protocol.

Reporting the maximum over 30 evaluations is optimistic by construction, and the concern it raises is whether the reported gain is an artifact of that selection. We therefore recomputed the same comparison from the stored evaluation curves under three protocols that involve no selection at all (Table IV). The absolute success rates do fall sharply—the eight-task mean drops from 78.0 to 62.6 under last-checkpoint reporting—which confirms that the protocol is optimistic in *level*. The gain, however, moves by at most one point, from +6.15 to +5.19, because both arms are selected by the identical rule and the selection bias largely cancels in the difference. The last-checkpoint row is the noisiest estimator available—a single 40-episode evaluation with no averaging—and its per-task sign pattern degrades accordingly (four of eight), whereas both averaging protocols retain seven of eight. We report best-checkpoint numbers in the main tables to stay comparable with the literature, and we take Table IV as the evidence that the conclusion does not rest on that choice.

Against Diffusion Policy, JEPA Policy is ahead on all eight tasks, for an eight-task mean of 84.2 against 75.7. The margin is dominated by Tool Hang, where this baseline reaches only 46.7; excluding that task the mean gap is +5.6 points, which is the figure we would rely on. The two methods are closest on Transport (98.3 vs. 96.7) and Square (95.8 vs. 93.3). **Inference cost.** Table III measures hardware latency and verifies the two forward passes; the second control pass skips the future projection. JEPA Policy averages 13.2 ms ($p_{95} = 14.5$ ms), using 3% of the 400 ms budget and sustaining 76 Hz. Relative to MIP, the future branch adds 0.79 M parameters, 0.29 ms (+2.2%), and 7 MiB of peak activation memory. The trained 100-step Diffusion Policy uses 100 verified calls and takes 439.5 ms (2.3 Hz), or $33\times$ longer, at a similar parameter count. Shortening it to 50/20/10/4 steps gives 223.0/94.4/47.3/21.5 ms (Fig. 6); these are timing-only settings, and four steps remain $1.6\times$ slower. Per-call costs are comparable (≈ 4.4 ms per denoising step and ≈ 6.6 ms per JEPA Policy pass); the difference is the call count. These single-task, single-seed timings establish only the latency scale.

TABLE II
BEST-CHECKPOINT SUCCESS RATE (%) ON THE EIGHT TASKS.

Task	MIP (action-only)	Dual-indep.	Dual-cross	Diffusion Policy	JEPA Policy
MokaMoka	86.7 85/90/85	85.8 82.5/87.5/87.5	82.5 77.5/87.5/82.5	84.2 85/77.5/90	92.5 92.5/95/90
MugMug	94.2 97.5/92.5/92.5	90.8 87.5/90/95	86.7 90/85/85	90.0 90/87.5/92.5	96.7 97.5/95/97.5
Square	94.2 95/92.5/95	94.2 95/92.5/95	94.2 97.5/95/90	93.3 92.5/95/92.5	95.8 95/97.5/95
Tool Hang	65.8 72.5/65/60	59.2 75/52.5/50	68.3 60/65/80	46.7 47.5/52.5/40	75.0 82.5/65/77.5
Transport (4cam)	97.5 97.5/97.5/97.5	99.2 97.5/100/100	97.5 97.5/97.5/97.5	96.7 95/95/100	98.3 97.5/97.5/100
Coffee Preparation	57.5 70/30/72.5	57.5 62.5/47.5/62.5	61.7 42.5/67.5/75	66.7 72.5/60/67.5	77.5 77.5/70/85
Kitchen	68.3 67.5/62.5/75	72.5 70/75/72.5	71.7 72.5/67.5/75	69.2 70/67.5/70	73.3 72.5/77.5/70
Three Piece Assembly	60.0 57.5/60/62.5	59.2 65/52.5/60	63.3 62.5/67.5/60	59.2 65/57.5/55	64.2 67.5/65/60
Mean (8 tasks)	78.0 80.3/73.8/80.0	77.3 79.4/74.7/77.8	78.2 75.0/79.1/80.6	75.7 77.2/74.1/75.9	84.2 85.3/82.8/84.4

Large number: mean over the three seeds {41, 42, 43}; small number: the three individual seeds in that order, matched across arms—the same seed index means the same data ordering and initialization stream. Bold values: best mean per task. The last row is the eight-task mean with its per-seed values. A few runs stopped short of 300k steps; in every arm the maximum occurs mid-run, so this does not affect a best-checkpoint reading.

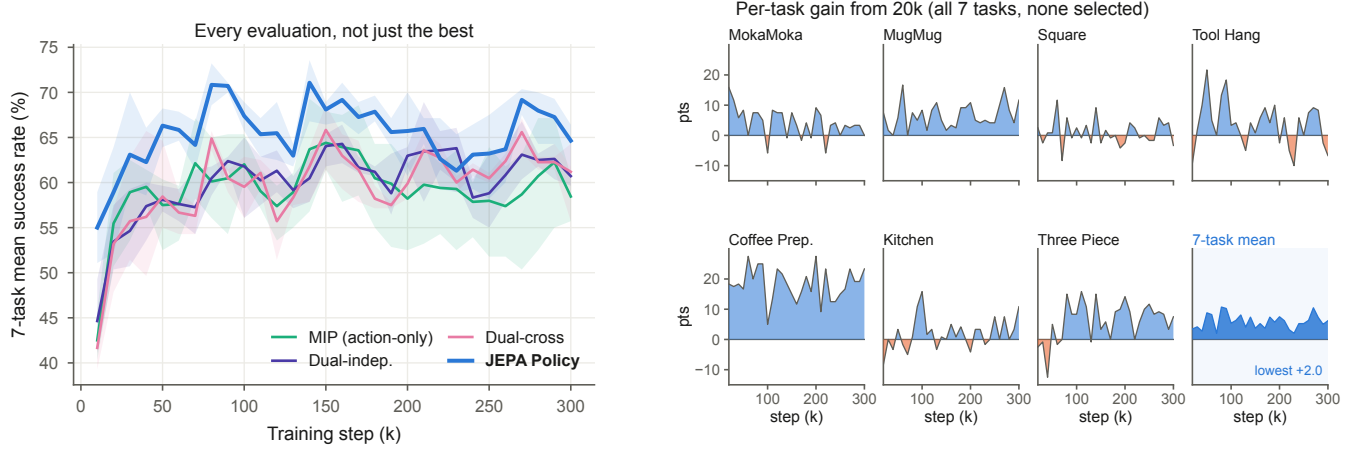


Fig. 5. Training curves for seven tasks with three completed seeds in all four arms. *Left*: seven-task mean across 30 evaluations (line: seed mean; band: seed range). *Right*: per-task JEPA Policy gain over MIP from the second evaluation onward; blue/orange denotes positive/negative gain on a shared scale. The seven-task mean is positive at all 29 evaluations (minimum +2.0); individual-task differences span -12.5 to $+27.5$ points.

TABLE III
Inference COST OF ONE POLICY DECISION (TOOL HANG, SEED 42, EMA WEIGHTS, BATCH SIZE 1) ON A PPU-ZW810E ACCELERATOR.

Method	Params (M)	Passes	Mean (ms)	p_{95} (ms)	Max Hz
MIP (action-only)	43.2	2	12.9	13.4	77.3
Diffusion Policy	42.6	100	439.5	462.6	2.3
JEPA Policy	44.0	2	13.2	14.5	75.6

Eager mode. JEPA Policy and MIP are timed interleaved in one PyTorch 2.6 process after 30 warmup and 300 synchronized iterations per arm; peak activation is 362/355 MiB (JEPA/MIP). Instrumented entry points verify “Passes.” The full audit path, including the second future projection, costs 12.8 ms. Diffusion Policy uses the same task, device, and aligned-protocol checkpoint in its trained 100-step DDPM configuration, but runs in a separate PyTorch 2.9 process. This increases uncertainty in its absolute latency, not the $33\times$ gap. The parameter counts are capacity-matched.

Training and evaluation cost. Training is where the future branch incurs cost. For the same 300,000 steps, it raises optimization time by 12–22% ($0.222 \rightarrow 0.271$ s/step on Tool Hang). The 30 evaluations add 2.7–5.2 h, up to one fifth of a run (Table V). A JEPA Policy step costs about $1.8\times$ a Diffusion Policy step (eight-task median; $1.5\text{--}2.1\times$ on the three shown) because it performs two passes and encodes the future. This higher optimization cost accompanies a $33\times$ cheaper

TABLE IV
SENSITIVITY OF THE EIGHT-TASK MEAN SUCCESS RATE (%) TO THE CHECKPOINT REPORTING PROTOCOL.

Protocol	Baseline	JEPA	Gain	Improved
Best of 30 evaluations	78.0	84.2	+6.1	8/8
Mean of last 5 evaluations	63.5	69.4	+5.8	7/8
Mean of all 30 evaluations	63.3	68.5	+5.2	7/8
Last checkpoint only	62.6	67.8	+5.2	4/8

Recomputed from the stored 30-evaluation curves of the same runs ($n = 3$ seeds). “Improved” counts tasks with a positive three-seed mean gain. The first row reproduces Table II.

decision. At one seed, JEPA Policy reaches a shared success target in fewer steps than each comparison arm on six of eight tasks, but the direction is not significant ($p = 0.29$). Every run uses one accelerator, takes about one day, and requires no video pretraining. The dual variants were about twice as slow in a different launch batch, so that observation is uncontrolled.

D. Shared Attention vs. Separate Branches

The central architectural claim of this paper is that the benefit of future prediction comes from placing action and future tokens in *one* shared attention stack, not from merely

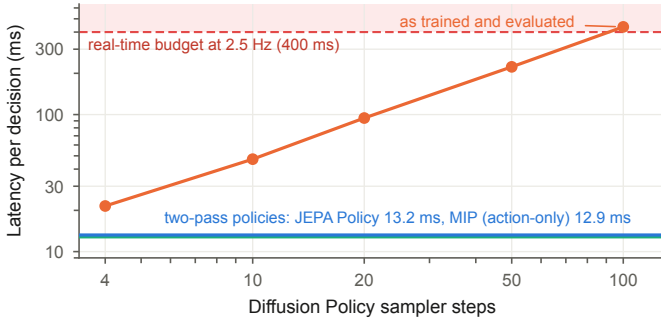


Fig. 6. Decision latency against Diffusion Policy sampler length (Tool Hang, same accelerator, batch 1, log-log). Shortened orange chains are timing-only; shading marks latency above the 400 ms budget. JEPA Policy and MIP form one blue band (13.2 vs. 12.9 ms).

TABLE V

TRAINING AND EVALUATION COST OF ONE RUN (300,000 GRADIENT STEPS, SEED 42) ON THE ACCELERATOR OF TABLE III.

Task	Arm	s/step	Train (h)	Eval (h)	Total (h)
Tool Hang	Diffusion Policy	0.136	11.3	—	—
	MIP (action-only)	0.222	18.5	4.8	23.3
	JEPA Policy	0.271	22.6	4.8	27.4
Square	Diffusion Policy	0.136	11.4	—	—
	MIP (action-only)	0.236	19.7	2.7	22.4
	JEPA Policy	0.289	24.0	2.7	26.7
MokaMoka	Diffusion Policy	0.206	17.2	—	—
	MIP (action-only)	0.273	22.7	5.2	27.9
	JEPA Policy	0.305	25.5	5.1	30.6

“Train” is the recorded run wall-clock minus the summed duration of the 30 periodic rollout evaluations, so it measures optimization only; “Eval” is that subtracted total; “Total” is the sum. “s/step” is the training column divided by 300,000. The Diffusion Policy rows come from its own logger, which records per-step optimization time but not evaluation duration, so its Eval and Total entries are left blank; being a different codebase, its s/step is indicative rather than a controlled measurement.

adding a future-prediction loss. Fig. 7 tests this directly with two isolated dual-branch implementations that keep the training recipe, losses, and parameter protocol identical and change only the topology. In particular, Dual-independent transplants the separate-branch topology of ACT-JEPA [6] into our training recipe while keeping everything else identical (same losses, MIP two-step training, stop-gradient targets instead of an EMA target encoder). It is therefore a controlled proxy for the separate-branch *design choice*, not a faithful reproduction of ACT-JEPA: differences in its original objective and EMA machinery are deliberately removed so that the comparison isolates the topology alone.

The result is that routing the future loss through a separate branch recovers none of the gain. Averaged over the eight tasks, Dual-independent reaches 77.3 and Dual-cross 78.2 against the MIP (action-only) arm’s 78.0 and JEPA Policy’s 84.2: the two dual variants land on the baseline, while the shared stack is 5.9–6.9 points above all three. JEPA Policy is the best variant on seven of the eight tasks; the exception is Transport (4cam), where Dual-independent reaches 99.2 against 98.3, a difference below one evaluation episode (2.5 points) on runs that did not complete the full protocol. Per task, Dual-independent is at or below the baseline on six of eight and Dual-cross on four of

eight, so the honest summary is that the separate-branch variants *match* the baseline rather than fall below it—adding the future loss outside the shared stack buys nothing, and even adding cross-attention between the branches (Dual-cross) does not recover the gain. Two properties of these variants sharpen what the comparison rules out. First, both have *more* trainable parameters than JEPA Policy (48.38 M and 53.73 M against 43.96 M, Sec. IV-B) and neither beats the MIP (action-only) arm, so the gain cannot be attributed to added capacity. Second, Dual-cross does make the two token groups mutually visible—through cross-attention at every layer—and still recovers nothing, so “the action and future tokens can attend to each other” is not by itself the operative condition; what distinguishes JEPA Policy is that the *same parameters* produce both readouts—Sec. IV-E measures the difference this makes to the gradient. Attaching a future prediction branch outside the shared stack is therefore not a cheaper substitute—the coupling of action and future tokens inside one Transformer is the load-bearing component, which also empirically separates JEPA Policy from separate-branch designs such as ACT-JEPA [6]. All cells are complete three-seed means except the two dual Transport (4cam) cells, whose runs stopped at 235k–243k of 300k steps; excluding Transport entirely leaves the ordering unchanged.

E. Where the Future Objective Acts

The ablation above shows *that* the topology matters; this subsection reports what the three topologies do to the gradient, which is the only channel through which the future objective can change the action parameters. We record, at every logged training step, the norm of the future-loss gradient on the parameters that produce the action readout, relative to the action-loss gradient on those same parameters.

Three facts separate the arms (Fig. 8). In Dual-independent the ratio is exactly zero at every step and every seed: the two decoder stacks share no parameters, so the future loss provably cannot move the action trunk. It does still reach the shared conditioning path, so this is a statement about the action decoder and not about the representation—which is what makes the arm a clean negative control for the topology rather than merely a weak one. In Dual-cross the ratio is non-zero but small, and it decays monotonically from 0.5% to 0.2%; cross-attention does connect the branches, but the connection thins out as training proceeds. In the shared stack the ratio rises to 2.9% in the 10–50k window and is still 0.9% at 300k—five to ten times Dual-cross at every window, with the three seeds so close that their range is barely visible. At convergence the same ordering holds on a fixed evaluation manifest: 0.68% for the shared stack against 0.135% for Dual-cross and exactly 0 for Dual-independent.

Two negative observations keep this from being over-read. First, mutual visibility alone does not explain the ranking: Dual-cross makes the action tokens visible to the future branch *more* strongly than the shared stack does—the action-loss gradient on its future tokens is about five times larger—and it still gains nothing, so the direction that matters is the future objective reaching the action parameters, not the reverse. Second, we do not report a gradient *alignment* statistic. The cosine between

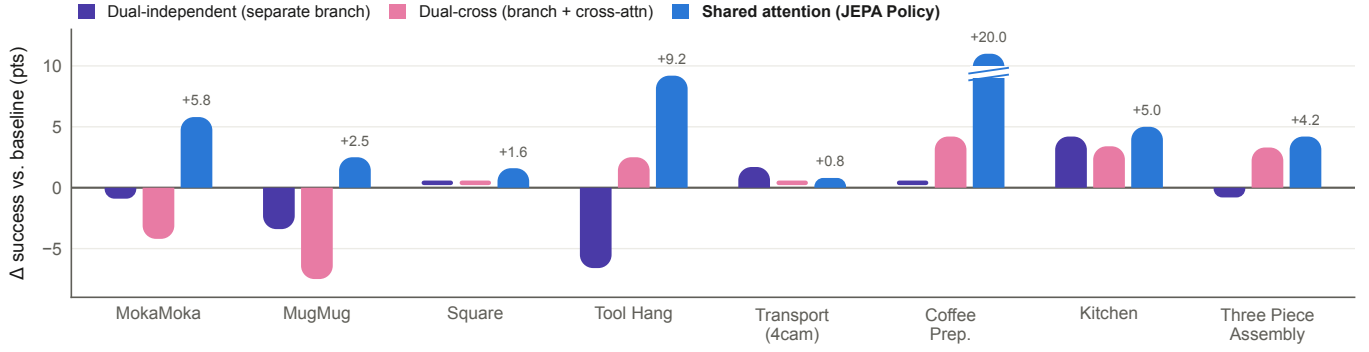


Fig. 7. Gain over MIP (three-seed mean best-checkpoint difference; per-task means in Table II). Dashes mark ties; the broken Coffee Preparation bar is +20.0; dual Transport runs are incomplete. JEPA Policy averages +6.1 points across eight positive tasks, versus -0.7 and $+0.2$ for the dual variants.

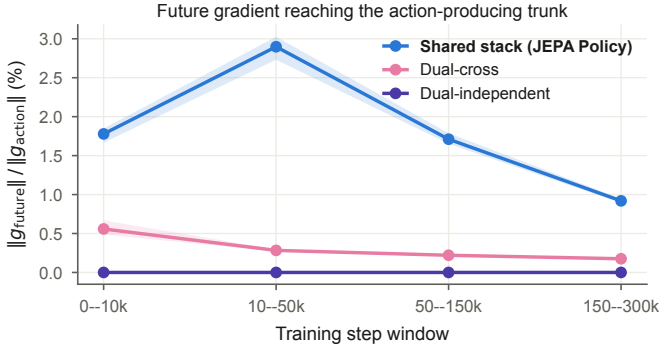


Fig. 8. Norm of the future-loss gradient reaching the trunk that produces the action, relative to the action-loss gradient on the same parameters, averaged over training windows (line: mean over seeds {41, 42, 43}; band: seed range; Tool Hang). In Dual-independent the quantity is exactly zero at every logged step, by construction. The shared stack stays an order of magnitude above Dual-cross throughout, peaking in mid-training rather than decaying.

the two gradients is small ($|\cos| < 0.03$) and its sign is not stable across measurement conditions, so we treat only the magnitude as evidence. The measurement is a share of gradient norm, not of the optimizer’s update. AdamW can amplify or attenuate a persistent component, so these values should not be read as update shares.

Cutting each route. The gradient shares above are correlational: they show that the future objective reaches the action parameters, not that the gain depends on it. We therefore severed each route directly and re-ran the full training and evaluation protocol at one seed (Table VI). Severing the backward route removes the entire gain: Back-cut averages 73.1 against 73.8 for the action-only policy at the same seed, so a future head whose gradient cannot reach the shared trunk buys nothing, while the full method reaches 82.8. Severing the forward route costs about half of it (78.4). Read together with Fig. 8, this is the interventional counterpart of the gradient measurement: the future objective acts mainly through the gradient it sends into the shared representation, and the action tokens’ direct view of the future token supplies the remainder.

Two cautions attach to this probe. It is a single seed, so we read the ordering and not the magnitudes; at 40 episodes per evaluation the single-seed granularity is 2.5 points, and Table II shows how wide the per-task seed spread can be.

TABLE VI
CUTTING EACH ROUTE BETWEEN THE FUTURE OBJECTIVE AND THE ACTION, AT A SINGLE SEED.

Task	MIP	Back-cut	Fwd-cut	JEPA Policy
MokaMoka	90.0	100.0	90.0	95.0
MugMug	92.5	95.0	92.5	95.0
Square	92.5	97.5	97.5	97.5
Tool Hang	65.0	17.5	55.0	65.0
Transport (4cam)	97.5	95.0	97.5	97.5
Coffee Preparation	30.0	65.0	70.0	70.0
Kitchen	62.5	62.5	72.5	77.5
Three Piece Assembly	60.0	52.5	52.5	65.0
Mean (8 tasks)	73.8	73.1	78.4	82.8

All four arms are the same seed (42) of the same code base, so the comparison is seed-matched; these numbers are therefore *not* comparable with the three-seed means of Table II. *Back-cut* keeps the forward route—action tokens still attend to the future token—but detaches the future loss from the shared trunk. *Fwd-cut* keeps the backward route—the future loss still shapes the shared representation—but removes the future token from the action tokens’ attention. Their Transport runs stopped at 270k and 230k of 300k steps (27 and 23 of the 30 evaluations); success on Transport saturates well before 100k, so this is unlikely to move those cells.

TABLE VII
FUTURE-LOSS RATIO AND HORIZON ABLATIONS (BEST-CHECKPOINT SUCCESS RATE %, THREE-SEED MEAN $\pm$ STD). The $\rho = 0.10$ row IS IDENTICAL TO THE $k = 4$ ROW.

Config.	MokaMoka	MugMug	Square	Tool Hang	Transport (4cam)
baseline	86.7 $\pm$ 2.9	94.2 $\pm$ 2.9	94.2 $\pm$ 1.4	65.8 $\pm$ 6.3	97.5 $\pm$ 0.0
$\rho = 0.05$	93.3 $\pm$ 5.2	92.5 $\pm$ 2.5	95.0 $\pm$ 0.0	70.8 $\pm$ 8.0	99.2 $\pm$ 1.4
$\rho = 0.10$ ($= k=4$)	92.5 $\pm$ 2.5	96.7 $\pm$ 1.4	95.8 $\pm$ 1.4	75.0 $\pm$ 9.0	98.3 $\pm$ 1.4
$\rho = 0.20$	91.7 $\pm$ 1.4	97.5 $\pm$ 2.5	97.5 $\pm$ 0.0	70.8 $\pm$ 8.0	99.2 $\pm$ 1.4
$k = 2$	90.0 $\pm$ 0.0	94.2 $\pm$ 2.9	95.0 $\pm$ 0.0	69.2 $\pm$ 13.8	99.2 $\pm$ 1.4
$k = 6$	93.3 $\pm$ 2.9	95.0 $\pm$ 2.5	98.3 $\pm$ 2.9	70.8 $\pm$ 6.3	100.0 $\pm$ 0.0

Back-cut’s parity with the baseline is not a mild degradation but the net of large offsetting swings—47.5 points on Tool Hang against +35.0 on Coffee Preparation, where the seed-42 baseline happens to be the collapsed run. We therefore claim only that severing the backward route destroys the aggregate gain and destabilizes the per-task behaviour, and we attach no significance statement to a single seed.

F. Future-Loss Ratio and Horizon

Table VII and Fig. 9 sweep the two hyperparameters of the future branch. No configuration with future supervision is

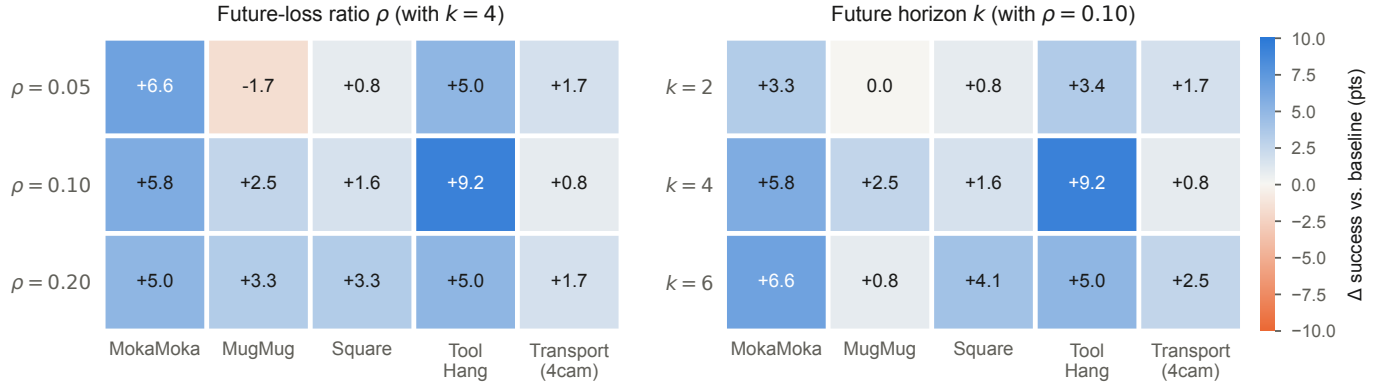


Fig. 9. Hyperparameter sensitivity as gain over the baseline (three-seed means, in points): future-loss ratio ρ (left) and future horizon k (right); blue is better than the baseline, orange worse. No configuration is materially worse than the baseline; $\rho \in [0.1, 0.2]$ and $k \in \{4, 6\}$ are uniformly strong choices.

TABLE VIII
CAMERA-CONFIGURATION SWEEP (BEST-CHECKPOINT SUCCESS RATE %, SINGLE SEED 42; DIRECTIONAL ONLY).

Config.	Moka	Mug	Square	Tool Hang	Coffee	Kitchen	Three Piece
Standard, baseline	90.0	92.5	92.5	65.0	30.0	62.5	60.0
Standard, future-4	95.0	95.0	97.5	65.0	70.0	77.5	65.0
Wrist only, baseline	62.5	80.0	80.0	27.5	37.5	42.5	15.0
Wrist only, future-4	72.5	87.5	80.0	30.0	47.5	47.5	17.5
Fixed only, baseline	87.5	70.0	90.0	35.0	67.5	65.0	42.5
Fixed only, future-4	—	—	87.5	37.5	62.5	62.5	40.0

Transport is omitted because the camera sources differ across rows; the fixed-only future-4 LIBERO cells are omitted because that batch was invalidated by a data bug.

materially worse than the baseline: across the five tasks and six settings, the only cell below the baseline is $\rho = 0.05$ on MugMug (92.5 vs. 94.2), a 1.7-point difference smaller than one evaluation episode. The method is otherwise insensitive within the swept range. $\rho = 0.10$ – 0.20 dominates $\rho = 0.05$ on the precision-heavy tasks (MugMug, Square, Tool Hang), and $k = 4$ – 6 dominates $k = 2$, consistent with the selection principle of Sec. III-A: too short a horizon makes the future target nearly redundant with the current observation. Tool Hang is the most sensitive task (best at $\rho = 0.10$, $k = 4$), while Square and Transport favor the larger $k = 6$.

G. Robustness to the Observation Configuration

Table VIII varies the camera configuration at seed 42. With wrist images only, six of seven tasks improve and one ties; MokaMoka and Coffee Preparation each gain 10.0 points. With fixed cameras only, all five usable differences remain within ± 5 points, so the gain is not reproduced. These results are directional: an identical single-seed configuration varied by 17.5 points between launch batches. We therefore infer no reversal from the fixed-camera results; the main comparison instead averages three seeds (Sec. IV-C).

H. Alternative Designs and Negative Results

Explicit isotropic regularization is fragile. Adding a SIGReg-style isotropic-Gaussian regularizer on the encoder

latents (weight 0.09; SIGReg is the sketched isotropic-Gaussian objective of LeJEPa [50]) to the full JEPa Policy destroys performance across all five tested tasks (Table IX); MokaMoka drops to 0.0 on all three seeds even though training itself completes normally. We tested a single weight, so this shows fragility at that setting rather than a property of explicit regularization in general; but it underlines that the action loss already provides the task-driven constraint the representation needs, without a sensitive extra regularizer.

State inputs. With low-dimensional state inputs instead of images (three seeds), the effect is task-dependent: Transport improves markedly ($52.5 \rightarrow 62.5$) while Tool Hang dips within one granularity step ($75.0 \rightarrow 72.5$). Future supervision thus systematically harms neither observation modality, and its benefit is not exclusive to image inputs.

I. Representation Collapse Audit

We audited the collapse question with a pre-specified spectral protocol rather than intuition. The audited quantity is spectral because the failure mode is not a constant output: methods that avoid *complete* collapse can still lose rank, part of the embedding spectrum falling to zero [21], which a test that outputs merely differ would miss. For seven tasks $\times$ three seeds, the formal JEPa Policy runs and matched action-only controls (42 runs; best and latest checkpoints; 84 checkpoints; 168 representation matrices over fixed 4,096-sample manifests) were scored in float64 with calibrated spectral metrics (effective rank, top-eigenvalue share, and tail energy TE32 against a pre-specified low-rank boundary of 0.05) and a fixed 256-candidate retrieval test for predictor input-dependence. TE32 is the share of centred spectral energy lying outside the leading 32 directions: writing $\lambda_1 \geq \dots \geq \lambda_D$ for the Gram eigenvalues of the mean-centred representation matrix,

TABLE IX
ADDING AN EXPLICIT SIGREG REGULARIZER (WEIGHT 0.09) TO JEPa POLICY (BEST-CHECKPOINT SUCCESS RATE %, THREE-SEED MEAN $\pm$ STD).

Config.	MokaMoka	Square	Tool Hang	Coffee	Three Piece
JEPa Policy	92.5 $\pm$ 2.5	95.8 $\pm$ 1.4	75.0 $\pm$ 9.0	77.5 $\pm$ 7.5	64.2 $\pm$ 3.8
+ SIGReg ($w=0.09$)	0.0 $\pm$ 0.0	42.5 $\pm$ 9.0	2.5 $\pm$ 0.0	12.5 $\pm$ 11.5	36.7 $\pm$ 2.9

$TE32 = 1 - \sum_{i \leq 32} \lambda_i / \sum_i \lambda_i$. Small values therefore indicate a spectrum concentrated in few directions, and the pre-specified boundary $TE32 = 0.05$ marks the point at which 95% of the centred energy has collapsed into 32 directions.

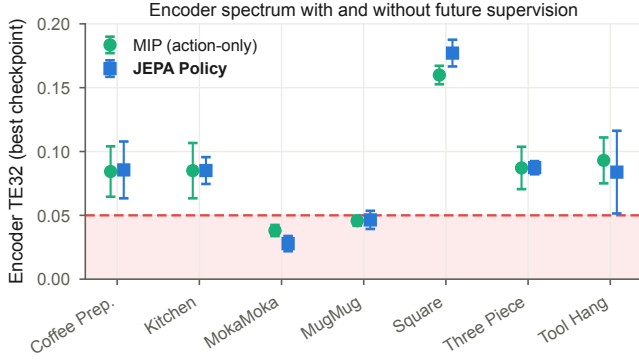


Fig. 10. Encoder TE32 at the best checkpoint (marker: three-seed mean; bar: worst seed; shading: below the pre-specified 0.05 boundary). JEPA Policy tracks MIP, including the MokaMoka and MugMug boundary exceptions.

The outcome supports a bounded claim: there is *no evidence of complete collapse or of systematic contraction to the pre-specified low-rank boundary*, and predictor retrieval is far above chance (input-dependent predictions); Fig. 10 shows representative spectra. The pre-specified task-level exception rule did trigger—the encoder TE32 of MugMug and MokaMoka, and the predictor TE32 of Tool Hang and Square, cross or touch the boundary at specific endpoints—so we explicitly do not claim the absence of low-rank contraction for all tasks. A complementary trajectory experiment (four 50k-step runs with snapshots at $\{0, 1k, 5k, 10k, 25k, 50k\}$ steps) shows the dynamics behind this: within the first 1,000 steps the fixed future loss drops by 97.7% while all seeds cross the low-rank boundary, after which TE32 *recovers* rather than continuing to contract—a sharp early contraction followed by partial recovery, not a monotone collapse; retrieval Recall@5 stays far above chance throughout (permutation $p = 1/1001$ at all endpoints).

Removing the action loss collapses the representation.

The audit above establishes only that collapse does not occur *with* the action loss. The $\lambda_a = 0$ variant of Sec. IV-B—verified to receive exactly zero action-loss gradient while the future loss still trains the encoder and shared trunk—was scored under the same protocol. The encoder representation contracts sharply: TE32 falls from 0.170 to 0.0014, some $36\times$ past the pre-specified 0.05 boundary and far beyond the contraction observed in the normal audited runs; centred effective rank falls from 181 to 24 of 384; and 92% of the embedding energy moves into the shared mean, leaving any two samples at cosine 0.918 against 0.021 for the matched normal run. It is not a complete collapse—predictor retrieval stays far above chance (Recall@1 0.976, permutation $p = 0.001$)—and that number is the mechanism rather than a reassurance: it is *higher* than the normal run’s 0.525, consistent with the future becoming nearly deterministic inside a 24-dimensional subspace. The future loss alone therefore admits a degenerate optimum and training finds it; the action loss is what removes it. The control covers one task and one seed.

TABLE X
ROLLOUT AUDIT: EARLY FUTURE-4 NMSE VS. FAILURE (24 CHECKPOINTS; 5,200 EPISODES). EPISODE-RESAMPLED CONFIDENCE INTERVALS ARE DESCRIPTIVE CONDITIONAL ON THESE CHECKPOINTS.

Task	Episodes	Failure rate	AUROC [95% CI]	PR-AUC
MugMug	600	14.5%	0.754 [0.693, 0.810]	0.377
Tool Hang	600	30.8%	0.722 [0.676, 0.766]	0.532
Coffee Preparation	600	38.8%	0.662 [0.617, 0.706]	0.566
Three Piece Assembly	600	47.7%	0.624 [0.578, 0.667]	0.588
Transport	1,000	7.3%	0.623 [0.566, 0.681]	0.127
Square	600	10.8%	0.620 [0.544, 0.694]	0.167
Kitchen	600	21.8%	0.572 [0.513, 0.629]	0.322
MokaMoka	600	16.7%	0.554 [0.488, 0.618]	0.254

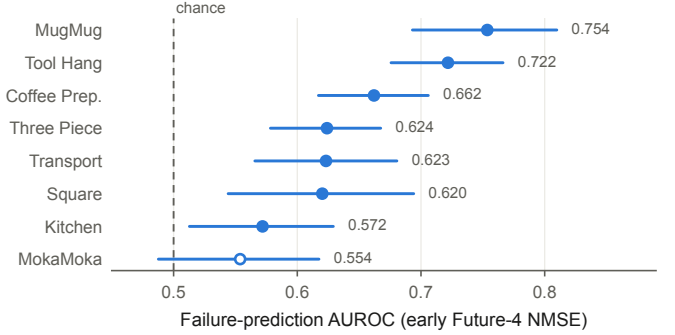


Fig. 11. Per-task failure-prediction AUROC of the early Future-4 NMSE with conditional episode-level 95% confidence intervals. The open marker (MokaMoka) denotes a confidence interval that crosses chance.

J. Future-Consistency Diagnostic at Rollout

We evaluated the deployment-time diagnostic of Sec. III-F on 24 frozen checkpoints (8 tasks $\times$ 3 seeds; checkpoint selection used no audit metric, removing selection bias) over 5,200 rollout episodes (4,040 successes, 1,160 failures); the evaluation pipeline was hard-gated beforehand (byte-identical episode evidence across worker counts after fixing a reset-seed bug). Each episode is scored by the median Future-4 NMSE (a 0.2 s-ahead prediction; Sec. IV-A) of its first five policy decisions, i.e., by whether the future prediction is already inconsistent *early* in the episode. For one decision the NMSE is the squared prediction error normalised by the root-mean-square magnitude of that sample’s target embedding, $NMSE = \|(\hat{\mathbf{z}}_{t+k} - \mathbf{z}_{t+k}) / \|\mathbf{z}_{t+k}\|_{RMS}\|_2^2 / d$, so that episodes are comparable within a task despite differing latent scales; an episode enters the analysis only if all five of its first decisions have a valid future target. The 95% intervals use 10,000 episode-level percentile-bootstrap resamples with a fixed seed. They are descriptive conditional on the three evaluated checkpoints per task, not checkpoint-population inference. Episode-level two-sided permutation tests use 10,000 resamples; checkpoint robustness is reported separately below.

The error ranks failures. Pooled per task, failures have higher early NMSE than successes on all eight tasks. AUROC exceeds 0.5 throughout; seven conditional episode-level intervals exclude chance (Table X, Fig. 11). MugMug (0.754) and Tool Hang (0.722) are strongest; only MokaMoka’s interval crosses chance. For MugMug, failures outrank successes in 75.4% of random failure–success pairs.

At the checkpoint level, 22 of 24 point estimates exceed 0.5 and 23 of 24 leave-one-seed-out tests agree in direction. Transport pooled results should be read cautiously: two of its three seeds are individually below 0.5, so the pooled value is not a stable single-checkpoint alarm.

Scoring under threshold-independent AUROC follows the protocol established for misclassification and out-of-distribution detection [39]; what differs is the cost, since the score falls out of a forward pass the policy already performs, whereas ensemble uncertainty needs M trained networks and M passes per decision [40].

Two controls rule out trivial explanations. Residualizing the NMSE against the realized latent visual-change magnitude leaves the signal unchanged (checkpoint-macro AUROC $0.6184 \rightarrow 0.6183$)—the association is not “failures just move more.” A persistence baseline that predicts the future latent to equal the current latent is near chance (0.508 vs. 0.618 for the learned predictor)—the signal comes from the learned prediction, not from generic latent motion.

Raw scores must be conditioned per task. Pooling raw NMSE across tasks destroys the signal entirely (AUROC 0.501, CI [0.484, 0.519]; permutation $p = 0.68$)—a Simpson-type effect of task-specific error scales. Any monitoring application must therefore calibrate per task (and for some tasks per checkpoint). Post-hoc Youden thresholds are descriptive only (chosen and evaluated on the same data): MugMug transfers best across seeds (held-out balanced accuracy 0.69–0.76), Coffee moderately, while Kitchen and MokaMoka support no useful raw threshold, and Tool Hang/Transport illustrate a ranking-vs-calibration gap (usable ordering, drifting absolute scale). We accordingly position the diagnostic as a per-task-calibrated risk signal for failure analysis and human hand-off, not a calibrated failure probability. It is learned from successful demonstrations and, unlike dedicated runtime monitors [42], [43], adds no foundation model, trains no scoring network, and draws no extra action samples. Operational thresholds nevertheless require held-out outcome-labeled rollouts. We ran no head-to-head comparison, so the distinction concerns cost and signal type, not detection accuracy.

K. Real-Robot Evaluation

We repeated the comparison on a physical two-arm ARX-5; one NVIDIA GeForce RTX 3090 ran inference from three 128×128 RGB streams, with the driver at 30 Hz and actions at 10 Hz. Across 3,048 steady-state JEPa Policy chunks from 160 episodes (first chunk excluded), frame-to-chunk latency was 20.1/63.3 ms (median/ p_{95}). MIP/DP-16/DP-100 recorded 27.6/67.5, 79.2/130.8 and 316.2/373.3 ms (median/ p_{95}). JEPa Policy/MIP executed eight actions per chunk; live profiles set one-/three-step compensation for DP-16/DP-100, leaving eight/six executable actions. The five tasks are Cabinet, Cup Stack, Cup Upright, Plate Grape, and Pen Insert (Fig. 12); their arm assignments are reported with Table XI. Each policy was evaluated at 60k, 100k, and 140k steps for 10 episodes per checkpoint, with operator-judged terminal success. Diffusion Policy used 16 and 100 denoising steps. All arms covered all tasks; the 63 sessions total 630 episodes without exclusions.

TABLE XI
REAL-ROBOT SUCCESS ON FIVE TASKS (THREE CHECKPOINTS PER ARM;
NOMINALLY 10 EPISODES PER CELL; THREE ACCIDENTAL REPEATS
POOLED).

Task	Steps	DP-100	DP-16	MIP	JEPa Policy
Cabinet	60k	0/10	0/10	2/10	5/10
	100k	0/10	0/10	3/10	6/10
	140k	0/10	0/10	4/10	7/10
	all	0%	0%	30%	60%
Cup Stack	60k	1/10	8/20	8/10	6/10
	100k	1/10	5/20	2/10	7/10
	140k	1/10	3/10	2/10	4/10
	all	10%	32%	40%	57%
Cup Upright	60k	1/10	2/10	4/10	5/10
	100k	5/10	3/10	8/10	14/20
	140k	2/10	7/10	6/10	10/10
	all	27%	40%	60%	72%
Plate Grape	60k	0/10	0/10	5/10	2/10
	100k	1/10	0/10	5/10	5/10
	140k	2/10	3/10	5/10	7/10
	all	10%	10%	50%	47%
Pen Insert	60k	4/10	6/10	8/10	10/10
	100k	3/10	8/10	10/10	10/10
	140k	7/10	9/10	10/10	9/10
	all	47%	77%	93%	97%
Pooled (episode-weighted)		18.7%	31.8%	54.7%	66.9%

DP-100 and DP-16 are Diffusion Policy at 100 and 16 denoising steps. The same two-arm platform runs every task; Cup Stack, Cup Upright and Pen Insert are performed with the right arm alone, Cabinet and Plate Grape with both. Three conditions were repeated by accident and both sessions are pooled, which is why those cells report 20 episodes: Cup Stack DP-16 at 60k and 100k, and Cup Upright JEPa Policy at 100k.

The pooled ordering carries over from simulation, with a larger margin. Table XI reports every cell. The episode-weighted pooled success rates are 66.9% for JEPa Policy, 54.7% for the MIP (action-only) arm, 31.8% for Diffusion Policy at 16 denoising steps and 18.7% at 100. Comparing matched task-checkpoint cells, JEPa Policy is ahead of Diffusion Policy in 14 of 15 cells at 16 steps and 15 of 15 at 100, never behind at either sampler length (mean margins +35 and +48 points; two-sided sign tests $p = 1.22 \times 10^{-4}$ and 6.10×10^{-5} , respectively). Because checkpoints within a task are correlated, these tests are descriptive consistency checks rather than independent-task inference. Against the action-only arm JEPa Policy is ahead in 9 of 15 cells, behind in 4 and level in 2, for a mean margin of +12.0 points that is not significant (sign test $p = 0.27$). The margin over Diffusion Policy is far larger here than in simulation, where it was +8.4 points and carried by a single task; the margin over the action-only arm is of the same order as the simulated +6.1 points, and it is concentrated in Cabinet (+30.0) while Plate Grape runs the other way (−3.3) and Pen Insert is close to a tie (+3.3, with both above 90%).

Cabinet separates the arms by failure stage. Neither Diffusion Policy configuration completes a single Cabinet episode, but the failures are not uniform. Taking an episode to have reached the final stage if it issued at least 194 actions—the lower quartile of the successful episodes on this task—7 of 30 DP-16 failures and 2 of 30 DP-100 failures had placed the object and failed only at the closing of the cabinet door,

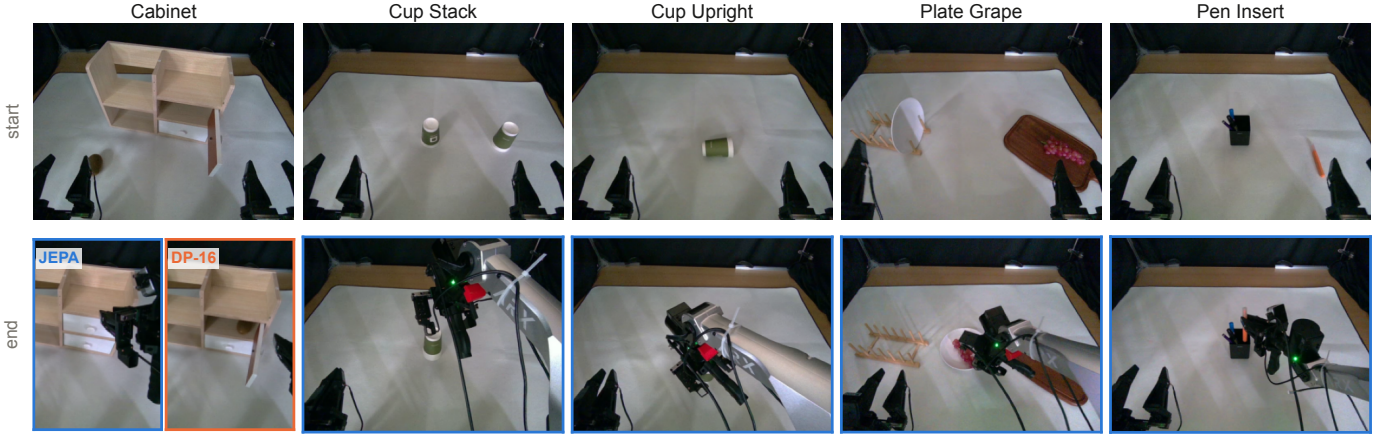


Fig. 12. Start and end frames from one successful JEPA Policy episode on each of the five real-robot tasks, including Pen Insert. The Cabinet end cell additionally contrasts the JEPA Policy success (door closed) with a DP-16 terminal failure (door left open). All frames are from the base camera; policies receive these views resized to 128×128 .

which inspection of the recorded video confirms (Fig. 12); the remaining failures occurred earlier, during approach or grasping. Door closure is a strict criterion for every arm: 7 of 21 MIP failures also stopped at that stage, and one MIP episode was scored a success only after three closing attempts. JEPA Policy failed at that stage in none of its 12 Cabinet failures—when it reached the door, it closed it. Plate Grape splits the same way: applying its own threshold of 254 actions, 3 of 15 action-only failures were terminal, against 0 of 16 for JEPA Policy and 0 of 27 for DP-16, whose failures on that task never reach the transfer at all.

V. DISCUSSION

The experiments support the central design claim: future supervision inside the action-generating stack improves manipulation success with little added latency. It adds only 0.29 ms (2.2%) to the two-pass MIP baseline; its 13.2 ms decision time is $33\times$ below Diffusion Policy as configured (Table III). The shared-vs-separate ablation is the sharpest evidence—the identical loss attached through a separate branch (with or without cross-attention) helps little or hurts, so the contribution is architectural, not a generic auxiliary-loss effect. The Dual-independent arm reproduces the property of ACT-JEPA [6] on which this test turns: a shared conditioning path trained by both losses, feeding two mutually invisible branches. Holding target modality, anti-collapse mechanism and training recipe fixed leaves topology as the only changed variable.

The results also bound the claim honestly. The lead over Diffusion Policy holds on all eight tasks but is carried by Tool Hang, and stands at +5.6 points without it. We therefore read the accuracy result as “at least as good as Diffusion Policy, and better on most tasks”, and place the weight of the contribution on the cost side, where the margin is a measured factor of 33. On collapse, the audit by itself permits only a bounded negative statement—no complete collapse and no systematic contraction to the pre-specified boundary, with named task-level low-rank exceptions—but the $\lambda_a = 0$ control supplies the positive half on one task: remove the action loss and the same protocol reports a boundary violation of roughly $36\times$, so

the constraint is doing work rather than merely being present. The SIGReg [50] result warns in the other direction, that a naive explicit regularizer at one weight can destroy the policy outright. The rollout audit shows the future branch carries real action-outcome information, but only as a *per-task-calibrated* risk signal. Raw scores do not transfer across tasks, and post-hoc thresholds remain descriptive.

Limitations. The characterisation is carried out in simulation and the hardware study is deliberately narrower: five tasks, three checkpoints, ten episodes per cell, one robot and one operator in a single laboratory. At that sample size a single cell is noisy—one condition repeated by accident returned 5/10 and 9/10 from the same checkpoint and the same scene—so the real-robot comparison with Diffusion Policy is consistent across cells (29 of 30 ahead and none behind), while the advantage over the action-only arm is directional only. An exploratory three-task RoboCasa development check, not included in the eight-task aggregate, showed no measurable effect of future supervision. This null result limits cross-suite generality and motivates a fully matched RoboCasa study rather than a universal claim. Table III isolates model time on one PPU, task, and seed with different PyTorch runtimes; RTX 3090 logs add shared-system end-to-end latency including sensing and middleware. Like-for-like comparison against few-step action generators, distilled [13] or single-stage [14], remains open. The best-checkpoint protocol, standard in this literature, is optimistic by construction, although Table IV shows the reported gain survives three selection-free protocols. Single-seed side studies are directional only, and multi-modal futures remain untested.

VI. CONCLUSION

We presented JEPA Policy, which couples expert actions with their demonstrated future representations in one shared Transformer and predicts both in two diffusion-free passes. Across eight simulated tasks, it improved the MIP (action-only) policy by 6.1 points and exceeded Diffusion Policy on every task, with action horizons matched on six, while reducing measured decision latency from 439.5 to 13.2 ms.

Dual-branch ablations isolated the shared stack as the load-bearing component. The 84-checkpoint collapse audit found no complete collapse with action supervision, while the 5,200-episode rollout audit supported only a per-task-calibrated risk signal. The five-task, 630-episode hardware study preserved the method ordering under the limits detailed above. Future work should broaden real-robot coverage, compare matched few-step generators [13], [14], validate diagnostic thresholds on held-out rollouts, and quantify multi-modal futures.

REFERENCES

- [1] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, 1988.
- [2] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [3] C. Pan *et al.*, "Much ado about noising: Dispelling the myths of generative robotic control," in *International Conference on Learning Representations (ICLR)*, 2026.
- [4] C. Chi *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems (RSS)*, 2023.
- [5] M. Assran *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [6] A. Vujanovic and A. Kovacevic, "ACT-JEPA: Novel joint-embedding predictive architecture for efficient policy representation learning," *IEEE Access*, vol. 14, pp. 78 895–78 906, 2026.
- [7] T. Z. Zhao *et al.*, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.
- [8] A. Mandlekar *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2021, pp. 1678–1690.
- [9] P. Florence *et al.*, "Implicit behavioral cloning," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2021, pp. 158–168.
- [10] N. M. M. Shafiuallah *et al.*, "Behavior transformers: Cloning k modes with one stone," in *Advances in Neural Information Processing Systems*, 2022.
- [11] Y. Song *et al.*, "Consistency models," in *International Conference on Machine Learning (ICML)*, 2023.
- [12] Y. Lipman *et al.*, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.
- [13] A. Prasad *et al.*, "Consistency policy: Accelerated visuomotor policies via consistency distillation," in *Robotics: Science and Systems (RSS)*, 2024.
- [14] K. Frans *et al.*, "One step diffusion via shortcut models," in *International Conference on Learning Representations (ICLR)*, 2025.
- [15] Y. Ze *et al.*, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," in *Robotics: Science and Systems (RSS)*, 2024.
- [16] Y. LeCun, "A path towards autonomous machine intelligence, version 0.9.2," OpenReview, 2022, position paper.
- [17] J.-B. Grill *et al.*, "Bootstrap your own latent - a new approach to self-supervised learning," in *Advances in Neural Information Processing Systems*, 2020.
- [18] A. Bardes, J. Ponce, and Y. LeCun, "VICReg: Variance-invariance-covariance regularization for self-supervised learning," in *International Conference on Learning Representations (ICLR)*, 2022.
- [19] J. Zbontar *et al.*, "Barlow twins: Self-supervised learning via redundancy reduction," in *International Conference on Machine Learning (ICML)*, 2021.
- [20] X. Chen and K. He, "Exploring simple Siamese representation learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 750–15 758.
- [21] L. Jing *et al.*, "Understanding dimensional collapse in contrastive self-supervised learning," in *International Conference on Learning Representations (ICLR)*, 2022.
- [22] Y. Tian, X. Chen, and S. Ganguli, "Understanding self-supervised learning dynamics without contrastive pairs," in *International Conference on Machine Learning (ICML)*, 2021.
- [23] H. Wu *et al.*, "Unleashing large-scale video generative pre-training for visual robot manipulation," in *International Conference on Learning Representations (ICLR)*, 2024.
- [24] Y. Tian *et al.*, "Predictive inverse dynamics models are scalable learners for robotic manipulation," in *International Conference on Learning Representations (ICLR)*, 2025.
- [25] A. Brohan *et al.*, "RT-1: Robotics transformer for real-world control at scale," in *Robotics: Science and Systems (RSS)*, 2023.
- [26] B. Zitkovich *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [27] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," in *Conference on Robot Learning (CoRL)*, 2024.
- [28] Octo Model Team *et al.*, "Octo: An open-source generalist robot policy," in *Robotics: Science and Systems (RSS)*, 2024.
- [29] Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic learning datasets and RT-X models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903.
- [30] K. Black *et al.*, " π_0 : A vision-language-action flow model for general robot control," in *Robotics: Science and Systems (RSS)*, 2025.
- [31] J. Sun *et al.*, "VLA-JEPA: Enhancing vision-language-action model with latent world model," in *European Conference on Computer Vision (ECCV)*, 2026.
- [32] S. Miao *et al.*, "JEPA-VLA: Video predictive embedding is needed for VLA models," *arXiv preprint arXiv:2602.11832*, 2026.
- [33] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.
- [34] M. Assran *et al.*, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.
- [35] M. Laskin, A. Srinivas, and P. Abbeel, "CURL: Contrastive unsupervised representations for reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2020.
- [36] M. Schwarzer *et al.*, "Data-efficient reinforcement learning with self-predictive representations," in *International Conference on Learning Representations (ICLR)*, 2021.
- [37] D. Hafner *et al.*, "Mastering diverse control tasks through world models," *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
- [38] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024.
- [39] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [40] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, 2017.
- [41] L. Lindemann *et al.*, "Safe planning in dynamic environments using conformal prediction," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 5116–5123, 2023.
- [42] C. Agia *et al.*, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Conference on Robot Learning (CoRL)*, 2024.
- [43] C. Xu *et al.*, "Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies," in *Robotics: Science and Systems (RSS)*, 2025.
- [44] B. Liu *et al.*, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.
- [45] A. Mandlekar *et al.*, "MimicGen: A data generation system for scalable robot learning using human demonstrations," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 1820–1864.
- [46] Y. Zhu *et al.*, "robosuite: A modular simulation framework and benchmark for robot learning," *arXiv preprint arXiv:2009.12293*, 2020.
- [47] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [48] K. He *et al.*, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [49] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.
- [50] R. Balestriero and Y. LeCun, "LeJEPA: Provable and scalable self-supervised learning without the heuristics," *arXiv preprint arXiv:2511.08544*, 2025.